

CICLOS

Repetir sin repetirte: while, for, break y continue

Ya sabes guardar datos y tomar decisiones. Ahora vas a enseñarle a la máquina lo único que hace mejor que cualquier humano: **repetir millones de veces sin cansarse y sin equivocarse**. Este es el tomo donde tu código deja de ser una lista de órdenes y se convierte en un motor.

- | | | | |
|----|---|----|--|
| 8 | while · repetir mientras se cumpla algo | 9 | for y range() · repetir un número exacto de veces |
| 10 | Los 4 patrones · contar, acumular, marcar, comparar | 11 | break y continue · el control fino del ciclo |
| 12 | Bucles anidados · ciclos dentro de ciclos | 13 | Ciclos que conversan · validación y menús |
| 14 | Depurar y proyecto · el jefe final | | |

Cómo está construido este tomo

Léelo una vez. Son tres minutos que te ahorran siete niveles de confusión.

Este tomo no está escrito para que lo leas: está escrito para que **lo ejecutes**. Cada nivel tiene la misma arquitectura, y esa repetición es deliberada — cuando tu cerebro ya sabe qué viene después, deja de gastar energía en orientarse y la invierte en entender.

01 | El mapa de los siete niveles

POR QUÉ EN ORDEN

Cada nivel usa lo del anterior como material de construcción. Saltarte uno no te ahorra tiempo: te lo cobra después, con intereses.

Los siete niveles forman una sola escalera. Los tres primeros te dan las dos herramientas de repetición y los patrones que resuelven casi todo; los tres siguientes te dan control fino; el último te convierte en alguien que *arregla* lo que otros solo rompen.

Nivel	Tema	Qué sabrás hacer al salir	Estado
8	while	Repetir mientras una condición sea verdadera y no morir en el intento	Empieza aquí
9	for y <code>range()</code>	Repetir un número exacto de veces y recorrer un texto letra por letra	Núcleo
10	Los 4 patrones	Contar, sumar, marcar y comparar: el 90 % de los ciclos del mundo real	Núcleo
11	break y continue	Salir antes de tiempo y saltarte vueltas sin romper nada	Control fino
12	Bucles anidados	Cuadrículas, tablas y dibujos; entender el costo de repetir dentro de repetir	Control fino
13	Ciclos que conversan	Validar lo que escribe el usuario y construir menús que no se caen	Aplicación
14	Depurar + proyecto	Cazar los 5 bugs clásicos de ciclos y entregar un programa completo	Jefe final

POR QUÉ COLORES

Cuando una información llega por dos canales a la vez (palabra + color), se recuerda mejor que por uno solo. Aquí el color siempre significa lo mismo: es una etiqueta, no un adorno.

En este tomo cada tipo de contenido tiene un color fijo y un ícono fijo. Después de dos niveles vas a reconocer un recuadro *antes* de leer su título, y eso te deja decidir en un segundo si lo lees ahora o lo dejas para el repaso.

Azul índigo · Concepto clave

Definiciones y estructura. Es lo que tendrías que subrayar si esto fuera papel.

Verde · Salida y logro

Lo que aparece en pantalla cuando todo salió bien, y las recompensas de cada nivel.

Ámbar · Error clásico

La trampa en la que cae todo el mundo. Leerlo antes te ahorra media hora de pelea.

Rojo · Peligro real

Cosas que cuelgan tu programa o lo hacen mentir en silencio. Poco frecuente, muy serio.

Magenta · Traza paso a paso

La simulación a mano, vuelta por vuelta. Es la herramienta central de este tomo.

Violeta · ¿Sabías qué?

Historia y cultura de la programación. Opcional para el examen, obligatorio para la conversación.

Turquesa · En la vida real

Dónde está esto funcionando ahora mismo en apps que usas todos los días.

Azul noche · Misión

El reto del nivel. Si no lo haces, no aprendiste el nivel: solo lo leíste.

1 · Escríbelo, no lo copies

Teclear cada ejemplo a mano es lo que instala la sintaxis en tus dedos. Copiar y pegar instala la ilusión de haber entendido.

2 · Predice antes de ejecutar

Antes de darle a ejecutar, escribe qué crees que va a salir. Acertar confirma; fallar enseña el doble. Los recuadros *Predice* existen para eso.

3 · Rómpelo a propósito

Cambia un número, quita una línea, invierte una condición. Un ciclo que se rompe frente a ti te enseña más que diez que funcionan.

4 · Explícalo sin mirar

El nivel es tuyo cuando puedes contarlo en voz alta con tus palabras. Si tienes que volver a la página, todavía no.



ANTES DE PASAR LA PÁGINA

Abre tu editor y déjalo abierto en una ventana al lado. Un tomo de ciclos leído sin ejecutar nada tiene aproximadamente el mismo efecto que ver videos de gimnasio.

04

Ruta de estudio: por qué no en un solo día

PRÁCTICA ESPACIADA

Estudiar 3 días de 40 minutos deja más huella que un día de 2 horas. El olvido parcial entre sesiones es justamente lo que fortalece el recuerdo.

Puedes leer los siete niveles en una tarde. No lo hagas. Los ciclos son el primer tema de programación donde el problema no es entender, sino **sostener varias cosas en la cabeza al mismo tiempo**: una variable que cambia, una condición que se revisa, un bloque que se repite. Eso necesita reposo entre sesiones.

Sesión	Contenido nuevo	Repaso obligatorio antes de empezar
Día 1 · 45 min	Niveles 8 y 9	Del tomo 1: <code>if</code> , <code>elif</code> , <code>else</code> y el operador <code>%</code>
Día 2 · 45 min	Niveles 10 y 11	Traza a mano el ejemplo 8.2 sin mirar la tabla de traza
Día 3 · 45 min	Niveles 12 y 13	Explica en voz alta la diferencia entre <code>break</code> y <code>continue</code>
Día 4 · 60 min	Nivel 14 + proyecto	Las tarjetas de repaso del final del tomo, tapando la respuesta
Día 10 · 20 min	—	Vuelve a hacer el reto del nivel que peor te salió. Ese es el repaso que cuenta

05

Chequeo previo: ¿estás listo?

Todo este tomo se apoya en cinco cosas del tomo anterior. Respóndelas de memoria. Si fallas dos o más, vuelve una hora al tomo 1: no es un retroceso, es afilar el hacha.



LO QUE YA DEBES TRAER PUESTO

1. ¿Qué devuelve `input()` siempre, sin importar lo que escriba la persona?
2. ¿Qué diferencia hay entre `=` y `==`?
3. ¿Qué significa `numero % 2 == 0` dicho en español?
4. ¿Qué le dice a Python que una línea está *dentro* de un `if`?
5. Escribe una f-string que muestre la variable `total` con dos decimales.

RESPUESTAS EN UNA LÍNEA

- Un `str` · comparar contra asignar · «¿el número es par?» · la indentación de 4 espacios · `f"{total:.2f}"`

while: repetir mientras se cumpla algo

El primer bucle de tu vida. La instrucción que convierte diez líneas aburridas en tres líneas inteligentes.

LO QUE DESBLOQUEARÁS

- 1 Escribir un `while` completo y explicar sus tres piezas obligatorias.
- 2 Simular a mano, vuelta por vuelta, cómo cambia una variable dentro de un ciclo.
- 3 Reconocer un bucle infinito antes de ejecutarlo, y saber cortarlo si ya pasó.
- 4 Decidir cuándo un problema pide un `while` y no otra cosa.

PALABRAS CLAVE

while bucle
iteración vuelta
condición contador
actualización
bucle infinito traza
Ctrl + C

8.1 | El problema que ningún `if` resuelve

DEL TOMO 1

Un `if` decide *si* algo pasa.
Un `while` decide *cuántas* veces pasa. Se parecen en la forma y hacen cosas opuestas.

Imagina que quieres mostrar en pantalla los números del 1 al 5. Con lo que sabes hasta hoy, la única salida es esta:

● a_la_fuerza.py

```
1 print("Vuelta 1")
2 print("Vuelta 2")
3 print("Vuelta 3")
4 print("Vuelta 4")
5 print("Vuelta 5")
```

Funciona. Y es una mala solución, por una razón que no es estética: **si mañana te piden mil vueltas, tu programa no crece — se derrumba**. Además, cada línea repetida es una línea donde puedes equivocarte, y si hay que cambiar la palabra «Vuelta» hay que cambiarla cinco veces (o mil).

Esta es la misma tarea escrita con un bucle. Léela despacio: cabe entera en cuatro líneas y funcionaría igual con cinco vueltas que con cinco millones.

con_bucle.py

```
1 vuelta = 1                # 1. INICIALIZAR: el punto de partida
2 while vuelta ≤ 3:         # 2. COMPROBAR: ¿seguimos?
3     print("Vuelta", vuelta) # CUERPO: lo que se repite
4     vuelta += 1           # 3. ACTUALIZAR: acercarse al final
5 print("Fin del bucle")    # fuera del bucle: se ejecuta una sola vez
```

SALIDA EN PANTALLA

```
Vuelta 1
Vuelta 2
Vuelta 3
Fin del bucle
```

CONCEPTO CLAVE

Bucle (o ciclo): una estructura que repite un bloque de código mientras se cumpla una condición. Cada repetición completa se llama **iteración** o, en cristiano, **vuelta**.

while: el bucle que repite *mientras* algo sea verdadero. No sabe de antemano cuántas vueltas dará: solo pregunta, antes de cada vuelta, si la condición sigue siendo cierta.

8.2 | Anatomía: el motor de tres tiempos

LA REGLA DEL 3

Si tu `while` falla, el 95 % de las veces es porque falta una de estas tres piezas.

Revisalas en orden antes de sospechar de cualquier otra cosa.

Todo `while` que funcione bien tiene tres piezas, y dos de ellas viven *fuera* de la palabra `while`. Por eso tantos principiantes se pierden: miran solo la línea del `while` cuando el problema está arriba o abajo.

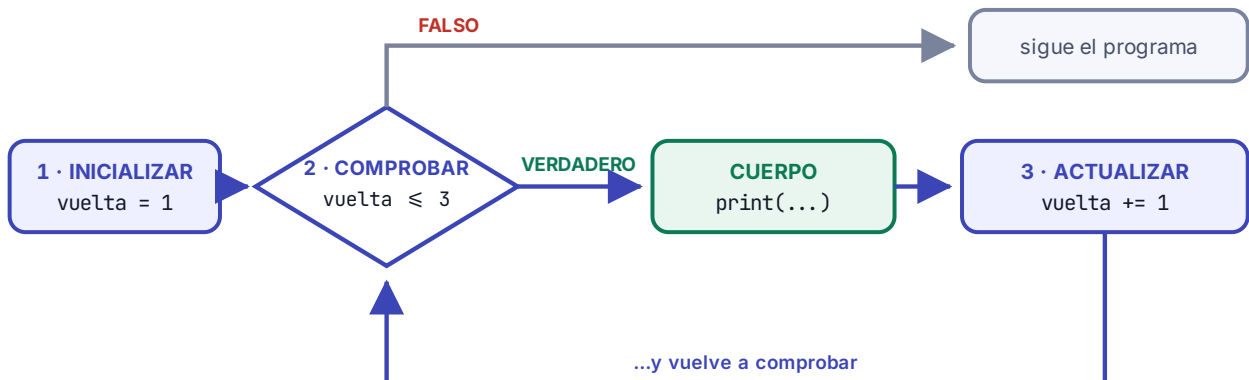


FIG. 8.1 – EL MOTOR DE TRES TIEMPOS. La condición se comprueba *antes* de cada vuelta, nunca en medio. Si la primera comprobación ya da falso, el cuerpo no se ejecuta ni una sola vez: un `while` puede dar cero vueltas.



PREDICE ANTES DE EJECUTAR

En el código de 8.1, cambia `vuelta = 1` por `vuelta = 9` y no toques nada más. ¿Cuántas líneas salen en pantalla? Escribe tu respuesta, ejecútalo y compara. Si dijiste «ninguna, y solo aparece *Fin del bucle*», ya entendiste la figura 8.1.

8.3

La traza: ver el bucle en cámara lenta

HERRAMIENTA CENTRAL

La traza no es un ejercicio escolar: es lo que hacen los programadores profesionales en su cabeza a toda velocidad. Tú la vas a hacer primero en papel, despacio.

Un ciclo es difícil al principio porque hay que sostener tres cosas a la vez en la cabeza. La solución es no sostenerlas: **escríbelas**. Una tabla de traza tiene una fila por vuelta y una columna por variable, más una columna para la respuesta de la condición.

TRAZA PASO A PASO · `con_bucle.py`

Vuelta	vuelta (antes)	¿vuelta <= 3?	Qué imprime	vuelta (después)
1. ^a	1	Sí	Vuelta 1	2
2. ^a	2	Sí	Vuelta 2	3
3. ^a	3	Sí	Vuelta 3	4
—	4	No	(sale del bucle)	4

Fíjate en la última fila: **siempre hay una comprobación de más**, la que falla y termina el ciclo. Por eso `vuelta` vale 4 al salir, aunque nunca se imprimió «Vuelta 4». Ese valor sobrante es la fuente número uno de errores de una unidad.

8.4

El bucle infinito: el peligro del nivel

Solo hay una forma de que un `while` se detenga: que su condición llegue a ser falsa. Si el cuerpo del bucle nunca cambia nada de lo que la condición mira, el programa repetirá lo mismo hasta que lo apagues. Compara estas dos versiones — la diferencia es *una sola línea*:

⚠ NUNCA TERMINA

```
1 vuelta = 1
2 while vuelta <= 3:
3     print("Vuelta", vuelta)
4 # falta actualizar: vuelta
5 # siempre valdrá 1
```

La condición mira `vuelta`, pero nadie cambia `vuelta`. La respuesta será «sí» para siempre.

✅ TERMINA BIEN

```
1 vuelta = 1
2 while vuelta <= 3:
3     print("Vuelta", vuelta)
4     vuelta += 1
5 # cada vuelta se acerca
6 # al final
```

Cada vuelta acerca la condición a ser falsa. Hay una **salida garantizada**.

⚠ SI TE PASA: CÓMO CORTARLO

Un bucle infinito no rompe tu computador y no borra nada: solo llena la pantalla o la deja congelada. Para detenerlo, haz clic en la consola y pulsa **Ctrl + C** (en Mac también **Ctrl + C**, no Cmd). Si estás en un intérprete del navegador, recarga la página.

Las tres causas, en orden de frecuencia: olvidaste la línea de actualización; actualizas una variable distinta a la que mira la condición (`i` contra `indice`); o la actualización va en el sentido contrario (`vuelta -= 1` cuando la condición esperaba que creciera).

⚠ ERROR CLÁSICO · LA ACTUALIZACIÓN FUERA DEL BLOQUE

Este es el más cruel, porque el código «se ve» bien:

```
while vuelta ≤ 3:
    print("Vuelta", vuelta)
vuelta += 1    ← sin sangría: está FUERA del bucle
```

La línea de actualización quedó a la izquierda, así que Python la considera parte del programa principal y no del ciclo. Resultado: bucle infinito. **En un ciclo, la indentación no es estilo: es la diferencia entre terminar y no terminar.**

8.5 | Un `while` que sí sirve para algo

CUÁNDO ELEGIR WHILE

Úsalo cuando *no sabes* cuántas vueltas hacen falta: hasta alcanzar una meta, hasta que el usuario escriba algo válido, hasta que se acabe la gasolina.

Los ejemplos que solo cuentan hasta 3 son buenos para aprender la mecánica y malos para entender el sentido. Aquí hay uno real: **¿cuántas semanas tienes que ahorrar para llegar a la meta?** Nadie sabe la respuesta de antemano — y eso es exactamente lo que hace que este problema pida un `while`.

● meta_de_ahorro.py

```
1 META = 100000          # constante: la meta no cambia
2 saldo = 0              # acumulador: empieza vacío
3 semanas = 0            # contador: empieza en cero
4
5 while saldo < META:     # mientras NO haya llegado
6     saldo += 15000      # ahorro semanal
7     semanas += 1        # una semana más
8     print(f"Semana {semanas}: llevas ${saldo:,}")
9
10 print(f"Meta alcanzada en {semanas} semanas.")
```

SALIDA EN PANTALLA

```
Semana 1: llevas $15,000
Semana 2: llevas $30,000
Semana 3: llevas $45,000
Semana 4: llevas $60,000
Semana 5: llevas $75,000
Semana 6: llevas $90,000
Semana 7: llevas $105,000
Meta alcanzada en 7 semanas.
```

Observa el detalle más elegante: el programa **calculó siete**, un número que no está escrito por ninguna parte. Ahí es donde un bucle deja de ser un ahorro de tecleo y se convierte en una herramienta de razonamiento. Cambia la meta a un millón y la respuesta aparece igual de rápido.



EN LA VIDA REAL · TODO LO QUE ESTÁ ENCENDIDO ESTÁ EN UN BUCLE

Tu videojuego favorito es literalmente un `while` gigante que da unas 60 vueltas por segundo: leer el control, mover todo, dibujar la pantalla, repetir. Se llama *game loop*, y cuando alguien dice que un juego «va a 60 FPS» está diciendo que ese bucle completa 60 vueltas cada segundo.

La barra de carga que miras mientras se descarga algo es un ciclo que repite «descarga un pedazo, actualiza el porcentaje» hasta que la condición `descargado < total` deja de cumplirse.

Y la app que tienes abierta ahora mismo está en un bucle esperando: «mientras el usuario no cierre, revisa si tocó la pantalla». Congelarse es, casi siempre, un bucle que se olvidó de terminar.



¿SABÍAS QUÉ?

Parece razonable pedirle a Python que avise cuando un bucle no va a terminar. Pues bien: **eso es matemáticamente imposible**, y no por falta de tecnología.

En 1936, **Alan Turing** demostró que no puede existir ningún programa capaz de decidir, para cualquier otro programa, si terminará o se quedará dando vueltas para siempre. Se conoce como el *problema de la parada*, y su demostración es de una elegancia brutal: si ese programa existiera, se podría construir con él otro que haga justo lo contrario de lo que el primero predice — una contradicción.

Lo asombroso es la fecha. Turing lo probó **antes de que existiera el primer computador**, razonando sobre una máquina imaginaria de papel. Así que la próxima vez que cuelgues la consola con un bucle infinito, ten claro que estás tropezando con uno de los límites conocidos de la

computación. **Ese límite es la razón de que la única defensa seas tú, revisando que cada vuelta acerque el final.**

🚩 MISIÓN DEL NIVEL 8

«La primera vuelta»

DIFICULTAD ●●○○○ · 30 MINUTOS · NECESITAS: EDITOR ABIERTO

- 1 Escribe un contador del 1 al 10 con `while`. Luego haz que cuente de **2 en 2**, y después al revés, del 10 al 1.
- 2 Cuenta regresiva de lanzamiento: imprime 5, 4, 3, 2, 1 en **un solo renglón** (recuerda `end=" "` del nivel 2) y termina con «¡Despegue!».
- 3 Adapta `meta_de_ahorro.py` a tu vida real: pon tu meta, tu ahorro semanal y responde cuántas semanas necesitas. **Prueba con un ahorro de 0** y explica qué pasó.
- 4 Haz la tabla de traza (en papel) de tu contador de 2 en 2. Cinco columnas, una fila por vuelta. Compárala con lo que imprime el programa: deben coincidir exactamente.
- 5 **Rómpelo a propósito:** borra la línea de actualización, ejecuta y córtalo con Ctrl + C. Anota cuántas líneas alcanzó a imprimir antes de que reaccionaras.

NIVEL JEFE: escribe un bucle que empiece en 100 y divida el valor entre 2 en cada vuelta, mientras sea mayor que 1. Antes de ejecutarlo, predice cuántas vueltas dará. ¿Por qué son tan pocas para un número tan grande? Esa intuición tiene nombre y la volverás a encontrar toda tu vida como programador.

Recompensa desbloqueada: repetición. Tu programa ya puede hacer mil veces lo que antes hacía una.

✓ CHECKPOINT · NIVEL 8

1. ¿Cuáles son las tres piezas de un `while` y cuál de ellas se olvida más?
2. ¿En qué momento exacto se comprueba la condición? ¿Puede un `while` dar cero vueltas?
3. Explica con tus palabras por qué en la traza de 8.3 la variable termina valiendo 4.

ESTE NIVEL EN 3 IDEAS

- Un `while` repite *mientras* su condición sea verdadera, y la revisa antes de cada vuelta.
- Inicializar, comprobar y actualizar: si falta la tercera, el bucle no termina nunca.

- La traza en papel convierte un ciclo confuso en una tabla que se entiende de una mirada.

ROJO

Copié el ejemplo pero no sabría escribir un `while` con la hoja tapada.

ÁMBAR

Lo escribo, pero todavía dudo dónde va la línea que actualiza.

VERDE

Escribo el bucle, predigo su traza y detecto un infinito de un vistazo.

for y range(): cuando sí sabes cuántas veces

El bucle que no se puede volver infinito por descuido, y la fábrica de números que lo alimenta.

LO QUE DESBLOQUEARÁS

- 1 Escribir un `for` y explicar qué hace su variable de control.
- 2 Usar `range()` en sus tres formas, incluido el paso negativo.
- 3 Recorrer un texto letra por letra sin escribir un solo número.
- 4 Elegir con criterio entre `for` y `while` ante un problema nuevo.

PALABRAS CLAVE

for in range()
variable de control
inicio fin excluido
paso recorrer
iterable
error de una unidad

9.1 | Tres líneas menos, cero bucles infinitos

REGLA DE ORO

¿Sabes de antemano cuántas vueltas? → `for`.
¿Depende de algo que pasará? → `while`.

El `while` del nivel anterior te obligaba a manejar tres piezas: inicializar, comprobar y actualizar. Cuando el número de vueltas se sabe desde el principio, ese trabajo es puro trámite — y todo trámite repetido es una oportunidad de equivocarse. Para eso existe `for`: **se encarga él solo de las tres piezas.**

● mismo_trabajo.py

```
1 # Con while: tú manejas el contador
2 vuelta = 0
3 while vuelta < 5:
4     print("Vuelta número", vuelta)
5     vuelta += 1
6
7 # Con for: Python maneja el contador por ti
8 for vuelta in range(5):
9     print("Vuelta número", vuelta)
```

SALIDA EN PANTALLA

```
Vuelta número 0
Vuelta número 1
Vuelta número 2
Vuelta número 3
Vuelta número 4
```

Las dos versiones imprimen exactamente lo mismo, pero la segunda tiene una propiedad que la primera jamás tendrá: **es imposible que se vuelva infinita**. No hay ninguna línea de actualización que puedas olvidar, porque no existe. Esa es la razón por la que, en el código profesional, la gran mayoría de los bucles son `for`.

CONCEPTO CLAVE

for: el bucle que recorre una colección de valores, uno por uno, de principio a fin. Se lee tal cual está escrito: «*para cada vuelta en este conjunto de valores, haz esto*».

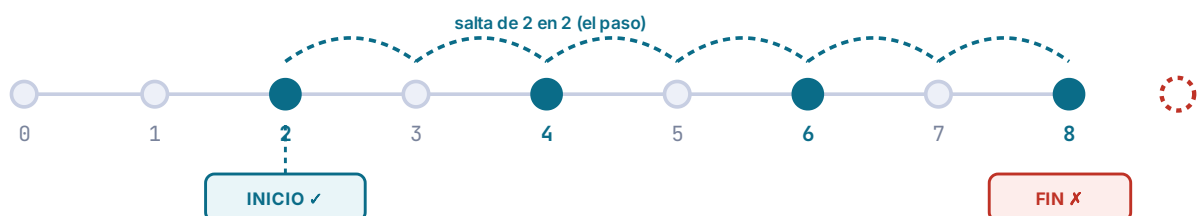
Variable de control: el nombre que va entre `for` e `in`. En cada vuelta, Python la crea de nuevo y le mete el siguiente valor. Tú no la creas ni la actualizas: solo la lees.

range(): una fábrica de números que produce una secuencia de enteros bajo demanda. Casi nunca se usa sola: existe para alimentar un `for`.

9.2 | range(): sus tres formas y su trampa

`range()` admite uno, dos o tres números, y significan cosas distintas. Todo el nivel se sostiene sobre una sola regla: **el número de inicio se incluye; el número final NO se incluye**. Esa asimetría es rara la primera vez y perfectamente lógica la décima.

`range(2, 9, 2)` produce 2, 4, 6, 8 · nunca llega al 9



El 9 marca dónde parar, no un valor que se vaya a usar. Piensa en él como la línea de meta: la pisas, no la cruzas.

FIG. 9.1 – LA FÁBRICA DE NÚMEROS. `range(inicio, fin, paso)` arranca en *inicio*, avanza de *paso* en *paso* y se detiene antes de *fin*.

● tres_formas_de_range.py

```
1 for i in range(3):           # solo el fin → empieza en 0
2     print(i, end=" ")
3 print()
4
5 for i in range(1, 6):        # inicio y fin → 1, 2, 3, 4, 5
6     print(i, end=" ")
7 print()
8
9 for i in range(0, 21, 5):     # inicio, fin y paso → de 5 en 5
10    print(i, end=" ")
11 print()
12
13 for i in range(10, 0, -2):   # paso negativo → cuenta hacia atrás
14    print(i, end=" ")
15 print()
```

SALIDA EN PANTALLA

```
0 1 2
1 2 3 4 5
0 5 10 15 20
10 8 6 4 2
```

⚠ ERROR CLÁSICO · EL ERROR DE UNA UNIDAD

Tiene nombre propio en inglés — *off-by-one* — porque es tan universal que hizo falta bautizarlo. Aparece de tres formas y todas te van a pasar:

- Quieres los números del 1 al 10 y escribes `range(1, 10)`: obtienes del 1 al **9**. Lo correcto es `range(1, 11)`.
- Quieres 5 vueltas y escribes `range(1, 5)`: obtienes 4. Con `range(5)` obtienes exactamente 5 vueltas — **ese es el motivo real de que el fin se excluya**: así `range(n)` siempre da n valores.
- Cuentas hacia atrás con `range(10, 0, -1)` y te preguntas dónde quedó el 0. Sigue siendo el fin excluido: si lo quieres, escribe `range(10, -1, -1)`.

Truco definitivo: cuando dudes, no razones — ejecuta `for i in range(...): print(i)` y míralo. Dos segundos contra veinte minutos de sospecha.

9.3 | Usar la variable de control

La variable del `for` no está de adorno: dentro del cuerpo vale algo distinto en cada vuelta, y ahí está casi todo el poder. Mira la tabla de multiplicar, que es el ejemplo perfecto porque el número que cambia es justo el que necesitas:

SOBRE EL NOMBRE I

Viene de *índice* y es una tradición de más de sesenta años. Es aceptable en bucles cortos; en bucles largos, un nombre con significado (`fila`, `alumno`) siempre gana.

● `tabla_de_multiplicar.py`

```
1  tabla = 7
2
3  print(f"--- TABLA DEL {tabla} ---")
4  for i in range(1, 11):
5      print(f"{tabla} x {i:2} = {tabla * i:3}")
```

SALIDA EN PANTALLA

```
--- TABLA DEL 7 ---
7 x  1 =  7
7 x  2 = 14
7 x  3 = 21
7 x  4 = 28
7 x  5 = 35
7 x  6 = 42
7 x  7 = 49
7 x  8 = 56
7 x  9 = 63
7 x 10 = 70
```

Los formatos `{i:2}` y `{tabla*i:3}` vienen del nivel 4: reservan un ancho fijo para que las columnas queden alineadas aunque los números tengan una o dos cifras. **Un bucle bien formateado se lee como una tabla; uno mal formateado se lee como una lista de accidentes.**



MICRO-RETO · 60 SEGUNDOS

Convierte esa tabla en una *tabla del revés*: que empiece en `7 x 10` y baje hasta `7 x 1`. Solo puedes tocar la línea del `range`. Pista: hay tres números y uno debe ser negativo.

9.4 | Recorrer un texto letra por letra

Aquí está la parte que sorprende a todo el mundo: `for` **no necesita números**. Puede recorrer cualquier cosa que tenga elementos en orden, y un texto es exactamente eso: una fila de caracteres. No hace falta `range`, ni contar, ni saber cuántas letras hay.

recorrer_texto.py

```
1 for letra in "PYTHON":
2     print(letra, end=" - ")
3 print()
4
5 # Contar vocales sin saber de antemano el largo de la palabra
6 palabra = input("Escribe una palabra: ")
7 vocales = 0
8
9 for letra in palabra.lower():
10     if letra in "aeiou":          # 'in' pregunta: ¿está dentro?
11         vocales += 1
12
13 print(f"'{palabra}' tiene {vocales} vocales.")
```

SALIDA EN PANTALLA

P - Y - T - H - O - N -

Escribe una palabra: Programacion

'Programacion' tiene 5 vocales.

OR DOS USOS DE `in` QUE NO SON LO MISMO

En la línea `for letra in palabra`, la palabra `in` forma parte de la estructura del `for`: significa «recorriendo».

En la línea `if letra in "aeiou"`, `in` es el operador de pertenencia del nivel 5: es una *pregunta* que devuelve `True` o `False`. Misma palabra, dos trabajos. El contexto (¿está en un `for` o en un `if`?) los distingue siempre.

9.5 | ¿Cuál de los dos elijo?

Si el problema suena así...	Usa	Por qué
«Repite esto 10 veces»	for	El número de vueltas está dado. No hay nada que decidir.
«Para cada letra de la palabra...»	for	Recorres algo que ya existe, de principio a fin.
«Hasta que el usuario escriba algo válido»	while	Podrían ser 1 o 40 intentos. Nadie lo sabe de antemano.
«Mientras quede batería / dinero / vida»	while	La condición depende de algo que cambia durante la ejecución.

«Del 1 al 100, saltando de 5 en 5»	for	Es exactamente lo que <code>range(1, 101, 5)</code> fabrica.
«Repite hasta que el resultado sea suficientemente preciso»	while	Es el caso de los cálculos científicos: se para al alcanzar la meta, no al llegar a un número de vueltas.

Si dudas: empieza con `for`. Solo cámbialo a `while` cuando descubras que no puedes decir cuántas vueltas necesitas.

⊕ EN LA VIDA REAL · EL BUCLE QUE PROCESA TU DÍA

Cuando subes 40 fotos a cualquier servicio, un `for` las recorre una por una: comprimir, generar miniatura, guardar. La misma línea de código funciona con 40 o con 4.000.

Cada barra de progreso honesta nace de un `for`: como el programa sabe cuántos elementos hay, puede calcular el porcentaje. Cuando ves una barra que se queda en 99 % eternamente, casi siempre es un `while` disfrazado de `for`.

Y los filtros de una foto recorren cada píxel con bucles como estos. Una imagen de 12 megapíxeles son 12 millones de vueltas... que tu teléfono resuelve en menos de un segundo.

💡 ¿SABÍAS QUÉ?

La pregunta que todo principiante hace en su primera semana: **¿por qué `range(5)` empieza en 0 y no en 1?**

En 1982, el informático neerlandés **Edsger Dijkstra** escribió una nota de dos páginas, hoy célebre, argumentando por qué numerar desde cero es la convención correcta. Su razonamiento es puramente estético y práctico a la vez: si un rango incluye el inicio y excluye el fin, entonces *la cantidad de elementos es simplemente la resta de los dos números*.

`range(0, 5)` tiene 5 elementos; `range(3, 7)` tiene 4. Sin sumar ni restar uno. Además, dos rangos consecutivos encajan sin dejar huecos ni solaparse: `range(0, 3)` y `range(3, 6)`.

Dijkstra escribía sus notas a mano, las numeraba con las siglas EWD y las repartía por correo entre colegas; se conservan más de 1.300. La costumbre de contar desde cero es anterior a él —viene de cómo se calculan las direcciones de memoria—, pero fue esa nota la que convirtió una decisión técnica en un argumento de diseño. **Cada vez que escribes**

`range(5)` estás usando una convención que alguien tuvo que defender por escrito.

🚩 MISIÓN DEL NIVEL 9

«Domar la fábrica de números»

DIFICULTAD ●●○○○ · 30 MINUTOS · ENTREGA: 5 PROGRAMAS CORTOS

- 1 Escribe cuatro `for` que impriman: los números del 1 al 20; los pares del 2 al 20; los múltiplos de 3 hasta 30; y una cuenta regresiva del 10 al 0 **incluyendo el 0**.
- 2 Pide un número con `input()` e imprime su tabla de multiplicar completa, alineada con formatos. Pruébalo con 12 y con 3: las columnas deben quedar cuadradas en ambos.
- 3 Pide una frase y cuenta cuántas letras **a** tiene. Luego amplíalo para que cuente también cuántos espacios — y descubre así cuántas palabras hay, aproximadamente.
- 4 Escribe el mismo programa dos veces: una con `for` y otra con `while`. Anota en una frase qué versión te pareció más clara y por qué.
- 5 **Predice y comprueba:** antes de ejecutar, escribe qué imprime `range(5, 5)` y qué imprime `range(5, 1)`. Después ejecútalos. Si no salió nada, no es un error: explica por qué tiene todo el sentido.

NIVEL JEFE: imprime la escalera `1, 2, 4, 8, 16...` hasta pasar de 1000. Aquí `range` no te sirve, porque su paso es una suma fija y tú necesitas multiplicar. Resuélvelo con `while` y explica en dos líneas por qué este problema pertenece al nivel 8 y no al 9.

Recompensa desbloqueada: repetición contada. Ya manejas las dos formas de repetir que existen en Python.

✅ CHECKPOINT · NIVEL 9

1. ¿Cuántas vueltas da `range(3, 12, 3)` y qué valores toma exactamente?
2. ¿Por qué un `for` no puede volverse infinito por descuido?
3. Explica los dos significados de la palabra `in` con un ejemplo de cada uno.

ESTE NIVEL EN 3 IDEAS

- `for` recorre valores uno por uno y gestiona el contador por ti: úsalo siempre que sepas cuántas vueltas necesitas.
- `range()` incluye el inicio y excluye el fin; el tercer número es el paso y puede ser negativo.

- Un `for` también recorre textos, letra por letra, sin usar números.

ROJO

Sigo sin ver la diferencia entre los tres números de `range()`.

ÁMBAR

Escribo el `for`, pero me equivoco por uno al fijar el final.

VERDE

Elijo entre `for` y `while` sin dudar y acierto el rango a la primera.

Contar, acumular, comparar y marcar

Cuatro esqueletos que resuelven la enorme mayoría de los ciclos que vas a escribir en tu vida. Apréndelos y dejarás de inventar desde cero.

LO QUE DESBLOQUEARÁS

- 1 Reconocer cuál de los cuatro patrones pide un problema con solo leerlo.
- 2 Colocar correctamente la variable acumuladora: fuera del bucle, siempre.
- 3 Calcular promedios, máximos y mínimos sin ninguna herramienta especial.
- 4 Combinar los cuatro patrones en un solo recorrido de datos.

PALABRAS CLAVE

- contador
- acumulador
- bandera
- centinela
- máximo
- mínimo
- inicialización
- ámbito
- promedio
- patrón

10.1 | Un esqueleto, cuatro variaciones

POR QUÉ PATRONES

Los expertos no resuelven cada problema desde cero: reconocen a cuál de un puñado de moldes conocidos se parece. Aprender los moldes es acortar años de experiencia.

Todos los ciclos útiles comparten la misma anatomía y solo cambian en tres detalles: qué variable preparas antes, qué le haces dentro y qué muestras al final.

1 · ANTES DEL BUCLE
preparas la variable

2 · DENTRO DEL BUCLE
la actualizas en cada vuelta

3 · DESPUÉS DEL BUCLE
la usas o la muestras

CONTADOR

total = 0

total += 1

print(total)

ACUMULADOR

suma = 0

suma += nota

print(suma / n)

FIG. 10.1 – EL ESQUELETO DE TODO PATRÓN. La variable siempre nace antes del bucle, cambia dentro y se lee después. Cambiar ese orden es el error del nivel.

10.2 | Patrón 1 · El contador

Responde a la pregunta «¿cuántos?». Se prepara en cero y sube de uno en uno cada vez que algo se cumple. Lo decisivo es que **el `+= 1` va dentro de un `if`**: si lo pones suelto, contarás todo en vez de contar lo que te interesa.

● contador.py

```
1 aprobadas = 0                                # nace en cero, FUERA del bucle
2
3 for i in range(1, 6):
4     nota = float(input(f"Nota {i}: "))
5     if nota >= 3.0:                            # solo cuenta si cumple
6         aprobadas += 1
7
8 print(f"Aprobaste {aprobadas} de 5 materias.")
```

10.3 | Patrón 2 · El acumulador

CONTADOR VS ACUMULADOR

El contador suma siempre **1** (cuenta cosas). El acumulador suma **el valor** (junta cantidades). Confundirlos da promedios absurdos.

Responde a «¿cuánto en total?». Es idéntico al contador salvo por una diferencia minúscula y enorme: en vez de sumar 1, suma el valor de la vuelta. Y con el total en la mano, el promedio sale gratis.

● acumulador.py

```
1 suma = 0                                    # el acumulador empieza vacío
2 CANTIDAD = 3
3
4 for i in range(1, CANTIDAD + 1):
5     nota = float(input(f"Nota {i}: "))
6     suma += nota                            # junta el VALOR, no un 1
7
8 promedio = suma / CANTIDAD                  # el promedio se calcula al final
9 print(f"Suma: {suma:.1f} · Promedio: {promedio:.2f}")
```

SALIDA EN PANTALLA

```
Nota 1: 4.0
Nota 2: 3.5
Nota 3: 5.0
Suma: 12.5 · Promedio: 4.17
```

⚠ EL ERROR MÁS CARO DEL NIVEL · LA VARIABLE QUE RENACE

Si declaras el acumulador *dentro* del bucle, se reinicia en cada vuelta y al final solo conserva el último valor. Lo peor: **no da ningún error**. El programa corre feliz y miente.

⚠ MIENTE EN SILENCIO

```
1 for i in range(3):
2     suma = 0
3     suma += 10
4 print(suma)    # 10
```

Cada vuelta borra lo acumulado. El total falso es **10**.

✅ CORRECTO

```
1 suma = 0
2 for i in range(3):
3     suma += 10
4 print(suma)    # 30
```

Nace una vez, crece tres veces. El total real es **30**.

Regla mnemotécnica: lo que se acumula, se prepara *antes*. Si la variable aparece a la misma altura que el `for`, va bien; si aparece corrida hacia la derecha, revísala dos veces.

10.4 | Patrón 3 · El campeón (máximo y mínimo)

Responde a «¿cuál es el mayor?». La idea es la de un torneo: guardas un campeón provisional y, cada vez que aparece alguien mejor, lo reemplazas. Al final del recorrido, el que quedó en pie es el ganador.

campeon.py

```
1 mejor = 0.0                # peor valor posible: cualquiera lo supera
2 peor = 5.0                 # mejor valor posible: cualquiera lo baja
3
4 for i in range(1, 4):
5     nota = float(input(f"Nota {i}: "))
6     if nota > mejor:        # ¿apareció alguien mejor?
7         mejor = nota       # nuevo campeón
8     if nota < peor:
9         peor = nota
10
11 print(f"Mejor nota: {mejor:.1f} · Peor nota: {peor:.1f}")
```

⚠ ERROR CLÁSICO · INICIALIZAR EL CAMPEÓN MAL

Empezar el máximo en `0` funciona con notas (nunca hay negativas), pero **falla estrepitosamente con datos que pueden ser negativos**: si buscas la temperatura máxima de una semana de invierno y todas son bajo cero, tu programa contestará 0, un valor que nunca existió.

La solución robusta: inicializa el campeón con *el primer dato real*, no con un número inventado. Pide el primer valor antes del bucle, guárdalo como campeón, y que el bucle recorra los demás.

10.5 | Patrón 4 · La bandera

NOMBRA BIEN TU BANDERA

Un booleano se nombra como una pregunta con respuesta sí/no: `hubo_error`, `esta_ordenado`, `encontrado`. Nunca `bandera1`.

Responde a «¿pasó al menos una vez?». Es una variable booleana que empieza en `False` y solo se levanta —nunca se vuelve a bajar— si ocurre lo que buscabas.

● bandera.py

```
1  hubo_perdida = False           # supongo que no, hasta ver una
2
3  for i in range(1, 6):
4      nota = float(input(f"Nota {i}: "))
5      if nota < 3.0:
6          hubo_perdida = True     # se levanta y ya no baja
7
8  if hubo_perdida:
9      print("Tienes al menos una materia por recuperar.")
10 else:
11     print("¡Periodo limpio! Ninguna materia perdida.")
```

10.6 | Los cuatro en un solo recorrido

Y aquí está el remate del nivel: **no hace falta un bucle por patrón**. Un solo recorrido puede alimentar los cuatro a la vez, porque cada uno solo necesita mirar el dato de esta vuelta. Este programa es, prácticamente, el corazón de cualquier informe de notas real.

● informe_completo.py

```
1 CANTIDAD = 5
2 suma = 0 # acumulador
3 aprobadas = 0 # contador
4 mejor = 0.0 # campeón
5 hubo_perfecta = False # bandera
6
7 for i in range(1, CANTIDAD + 1):
8     nota = float(input(f"Nota {i}: "))
9     suma += nota
10    if nota >= 3.0:
11        aprobadas += 1
12    if nota > mejor:
13        mejor = nota
14    if nota == 5.0:
15        hubo_perfecta = True
16
17 promedio = suma / CANTIDAD
18 print(f"\nPromedio: {promedio:.2f}")
19 print(f"Aprobadas: {aprobadas} de {CANTIDAD}")
20 print(f"Mejor: {mejor:.1f}")
21 print(f"¿Alguna 5.0? {'Sí' if hubo_perfecta else 'No'}")
```

SALIDA EN PANTALLA

Nota 1: 4.5
Nota 2: 2.8
Nota 3: 3.9
Nota 4: 5.0
Nota 5: 3.0

Promedio: 3.84
Aprobadas: 4 de 5
Mejor: 5.0
¿Alguna 5.0? Sí

TRAZA PASO A PASO · CUATRO VARIABLES A LA VEZ

Vuelta	nota	suma	aprobadas	mejor	hubo_perfecta
inicio	—	0	0	0.0	False
1.ª	4.5	4.5	1	4.5	False
2.ª	2.8	7.3	1	4.5	False
3.ª	3.9	11.2	2	4.5	False
4.ª	5.0	16.2	3	5.0	True
5.ª	3.0	19.2	4	5.0	True

Míralo columna por columna, no fila por fila: cada columna es un patrón viviendo su propia vida. **La de aprobadas se queda quieta en la vuelta 2** (2.8 no cumple) y la de **mejor** solo cambia dos veces. Esa lectura vertical es exactamente lo que hay que aprender a ver.



PREDICE ANTES DE EJECUTAR

Si las cinco notas fueran `2.0, 2.0, 2.0, 2.0, 2.0`, ¿qué valdría **mejor** al final? ¿Y si fueran todas `0.0`? La segunda respuesta revela por qué el recuadro ámbar de 10.4 importa tanto.



EN LA VIDA REAL · LOS CUATRO PATRONES EN TU PANTALLA

Contador: los «me gusta» de una publicación, los días de racha de una app de idiomas, las notificaciones sin leer.

Acumulador: el total de tu carrito de compras, los kilómetros de la semana en una app de running, las horas de pantalla del informe mensual.

Campeón: el récord personal, la publicación más vista del mes, la temperatura máxima del día.

Bandera: ese punto rojo que aparece cuando hay *algún* mensaje nuevo. No dice cuántos: solo se levantó porque hubo al menos uno.



¿SABÍAS QUÉ?

La palabra **acumulador** no la inventaron los profesores de programación: es una pieza física.

En los primeros computadores, el procesador tenía un registro especial — una casilla de memoria minúscula, directamente cableada a la unidad

aritmética— llamado literalmente *acumulador*. Casi todas las operaciones lo usaban por defecto: sumar significaba «suma esto *al acumulador*». Máquinas legendarias como el PDP-8 o el procesador 6502 (el del Commodore 64 y las consolas de los ochenta) se programaban prácticamente girando en torno a ese único registro.

Cuando escribes `suma += nota`, Python traduce eso a instrucciones que, muchas capas más abajo, siguen haciendo lo mismo que aquella pieza de metal. **El patrón que acabas de aprender no es una convención escolar: es la forma en que las máquinas piensan desde que existen.**

🚩 MISIÓN DEL NIVEL 10

«El informe que sirve»

DIFICULTAD ●●●○○ · 40 MINUTOS · ENTREGA: UN PROGRAMA + UNA TRAZA A MANO

- 1 Escribe `informe_completo.py` a mano y ejecútalo con tus notas reales. Añade una línea que muestre **cuántas materias perdiste** (contador nuevo, no restes).
- 2 Agrégale el mínimo con el patrón campeón. Prueba con todas las notas iguales y comprueba que mejor y peor coincidan.
- 3 Haz un **contador de gastos**: pide 7 gastos diarios, muestra el total, el promedio, el día más caro y una bandera que avise si algún día pasaste de cierto límite.
- 4 Haz la traza a mano de tu programa de gastos con 3 días inventados. Debe coincidir exactamente con la salida real. **Si no coincide, el error está en tu cabeza o en tu código: encontrarlo es el ejercicio.**
- 5 Escribe un contador de vocales **y** uno de consonantes sobre una frase, en un solo recorrido. Verifica que la suma de ambos más los espacios dé el largo total.

NIVEL JEFE: calcula el promedio *sin usar* una constante con la cantidad. Es decir: que el usuario diga cuántas notas va a meter y el programa se adapte. Ahora tienes dos variables que crecen en paralelo — suma y cantidad — y el promedio depende de las dos. Explica por qué dividir entre una constante equivocada es un error lógico y no de sintaxis.

Recompensa desbloqueada: memoria de recorrido. Tus ciclos ya no solo repiten: recuerdan lo que vieron.

✅ CHECKPOINT · NIVEL 10

1. ¿Cuál es la única diferencia de escritura entre un contador y un acumulador, y qué consecuencia enorme tiene?

2. ¿Por qué inicializar el máximo en `0` puede producir una respuesta falsa?
3. ¿Qué pasa exactamente si escribes `suma = 0` dentro del bucle, y por qué Python no protesta?

ESTE NIVEL EN 3 IDEAS

- Contar, acumular, comparar y marcar: cuatro moldes que cubren casi todos los ciclos reales.
- La variable se prepara antes del bucle, se actualiza dentro y se usa después. Siempre en ese orden.
- Un solo recorrido puede alimentar los cuatro patrones a la vez: no necesitas cuatro bucles.

ROJO

Todavía confundo cuándo sumar 1 y cuándo sumar el valor.

ÁMBAR

Uso los patrones, pero a veces pongo la inicialización en el sitio equivocado.

VERDE

Leo un enunciado y sé de inmediato qué patrones necesita y cuántos bucles.

break, continue y el else que nadie espera

Hasta ahora tus bucles empezaban y terminaban solos. Aquí aprendes a interrumpirlos, a saltarte vueltas y a saber si llegaron al final.

🕒 LO QUE DESBLOQUEARÁS

- 1 Cortar un bucle en seco con `break` en cuanto encuentres lo que buscabas.
- 2 Saltarte una vuelta con `continue` sin abandonar el ciclo.
- 3 Usar el patrón profesional `while True` + `break` sin miedo.
- 4 Aprovechar el `else` de los bucles, una rareza que Python casi no comparte con ningún otro lenguaje.

🔑 PALABRAS CLAVE

`break` `continue`

`while True`

`else de bucle`

`salida temprana`

`búsqueda` `filtro`

`bucle controlado`

`eficiencia`

Sí, existen los `dos`, y son exactamente lo que su nombre sugiere: `break` rompe el bucle y `continue` continúa con la siguiente vuelta. Son dos palabras, se escriben solas en una línea y cambian por completo lo que puedes construir.

11.1 |

break: la salida de emergencia

REGLA DEL ALCANCE

`break` abandona únicamente el bucle donde está escrito. Si hay un bucle dentro de otro, sale del interior y el de afuera sigue. Lo verás en el nivel 12.

Cuando `break` se ejecuta, el bucle termina *en ese instante*: no se completa la vuelta, no se comprueba la condición, no se ejecuta nada más del cuerpo. El programa salta a la primera línea que haya después del bucle.

Su uso natural es la **búsqueda**: en cuanto encuentras lo que buscabas, seguir buscando es desperdicio puro.

● buscar_y_parar.py

```
1 for numero in range(101, 200):
2     if numero % 13 == 0:
3         print(f"Encontrado: {numero}")
4         break                    # ya está: no tiene sentido seguir
5     print(f"{numero} no sirve")
6
7 print("El bucle terminó.")
```

SALIDA EN PANTALLA

```
101 no sirve
102 no sirve
103 no sirve
Encontrado: 104
El bucle terminó.
```

Sin `break`, ese bucle habría dado **99 vueltas** en vez de 4, y habría impreso 95 líneas inútiles. Con datos pequeños es una molestia; con un millón de registros es la diferencia entre un programa que responde y uno que se queda pensando.

11.2 | continue: saltarse esta vuelta

`continue` es más suave: no termina el bucle, solo abandona *la vuelta actual* y va directo a la siguiente. Todo lo que estuviera escrito después del `continue` en esa vuelta, sencillamente no ocurre.

Su uso natural es el **filtro**: descartar los casos que no te interesan antes de hacer el trabajo pesado.

● filtrar_y_seguir.py

```
1 for numero in range(1, 11):
2     if numero % 2 == 0:
3         continue           # los pares no me interesan: siguiente
4     print(numero, end=" ")
5
6 print("\n--- solo los impares ---")
```

SALIDA EN PANTALLA

```
1 3 5 7 9
--- solo los impares ---
```

break — se acabó el bucle



continue — solo se salta esta vuelta

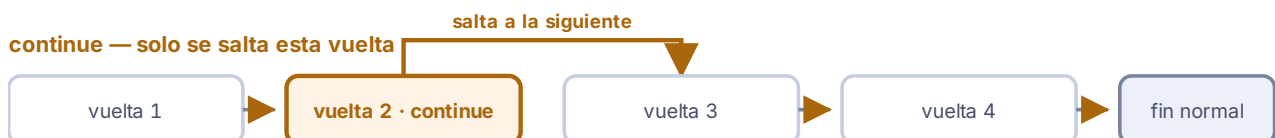


FIG. 11.1 — DOS INTERRUPCIONES MUY DISTINTAS. `break` abandona el bucle entero; `continue` abandona solo la vuelta en curso. Confundirlos produce programas que hacen la mitad del trabajo... o ninguno.

⚠ PELIGRO REAL · CONTINUE DENTRO DE UN WHILE

En un `for`, `continue` es inofensivo: Python avanza el contador por ti. En un `while`, en cambio, **si el `continue` se ejecuta antes de la línea que actualiza la variable, esa actualización nunca ocurre** — y tienes un bucle infinito perfecto.

```
i = 0
while i < 5:
    if i == 2:
        continue    ← se va sin ejecutar el i += 1
    print(i)
    i += 1           ← nunca se alcanza cuando i vale 2
```

Imprime 0, 1 y se queda colgado para siempre en el 2. **Regla de supervivencia:** si usas `continue` en un `while`, actualiza la variable *antes* del `continue`. O mejor: usa un `for`.

11.3 | while True + break: el patrón profesional

NO ES HACER TRAMPA

Un `while True` con un `break` claro es más legible que una condición complicada. Lo que está prohibido es un `while True` *sin salida*.

A veces la condición de salida es incómoda de escribir por adelantado: depende de algo que solo sabrás en mitad de la vuelta. La solución que usa todo el mundo es declarar el bucle eterno y poner la salida donde de verdad corresponde.

`True` siempre es verdadero, así que `while True:` significa literalmente «repite para siempre». Lo que lo convierte en código correcto y no en un accidente es el `break`: **el compromiso que asumes al escribir `while True` es garantizar al menos una salida real.**

● control_de_acceso.py

```
1 CLAVE = "python2026"
2 intentos = 0
3
4 while True:
5     clave = input("Clave: ")
6     intentos += 1
7
8     if clave == CLAVE:
9         print("Acceso concedido.")
10        break                    # salida 1: acertó
11
12    if intentos == 3:
13        print("Demasiados intentos. Cuenta bloqueada.")
14        break                    # salida 2: se agotó
15
16    print(f"Clave incorrecta. Intentos restantes: {3 - intentos}")
```

SALIDA EN PANTALLA

```
Clave: hola
Clave incorrecta. Intentos restantes: 2
Clave: python
Clave incorrecta. Intentos restantes: 1
Clave: python2026
Acceso concedido.
```



PREDICE ANTES DE EJECUTAR

¿Qué pasa si borras el segundo `break` (el del límite de intentos)? ¿Y si borras los dos? Escribe tu predicción, pruébalo y ten el Ctrl + C listo.

11.4 | El `else` de los bucles

Esta es una de las rarezas más bonitas de Python, y también una de las peor nombradas. Un bucle puede llevar un `else` al final, alineado con el `for` o el `while`, y la regla es exactamente esta:

El bloque `else` se ejecuta si el bucle terminó por su cuenta. Si salió por un `break`, el `else` se salta.

Dicho de otro modo: `else` significa aquí «*si nunca hubo break*». Y eso resuelve de un plumazo el problema clásico de toda búsqueda: **¿cómo sé que NO encontré nada?** Sin `else` tendrías que usar una bandera; con él, no.

● buscar_letra.py

```
1 palabra = input("Palabra: ")
2 buscada = input("Letra a buscar: ")
3 posicion = 0
4
5 for letra in palabra:
6     posicion += 1
7     if letra == buscada:
8         print(f"Encontrada en la posición {posicion}")
9         break
10 else:
11     print(f"La letra '{buscada}' no aparece en '{palabra}'")
```

SALIDA EN PANTALLA

```
Palabra: murcielago
Letra a buscar: i
Encontrada en la posición 5

Palabra: murcielago
Letra a buscar: z
La letra 'z' no aparece en 'murcielago'
```

⚠ ERROR CLÁSICO · LEER MAL EL `else`

Casi todo el mundo lo interpreta al revés la primera vez, porque el `else` de un `if` significa «si no se cumplió». Aquí no significa eso.

- **NO** quiere decir «si el bucle no se ejecutó».
- **NO** quiere decir «si la condición fue falsa».
- **SÍ** quiere decir «si el bucle llegó al final sin encontrarse un `break`».

Muchos programadores veteranos opinan que debería haberse llamado `nobreak`. Léelo así mentalmente y no volverás a equivocarte.

11.5 | Los cuatro comportamientos, uno al lado del otro

Palabra	Qué hace exactamente	Cuándo la usas
break	Termina el bucle completo, ahí mismo.	Encontraste lo que buscabas; el usuario pidió salir; se agotaron los intentos.
continue	Abandona la vuelta actual y pasa a la siguiente.	Este dato no te interesa o no es válido, y no quieres anidar más <code>if</code> .
else (de bucle)	Se ejecuta solo si NO hubo <code>break</code> .	Avisar que una búsqueda terminó sin resultado.
pass	No hace absolutamente nada.	Marcar un bloque que todavía no escribiste, para que Python no proteste por el hueco.

🌐 EN LA VIDA REAL · INTERRUPCIONES POR TODAS PARTES

Buscar en un documento (Ctrl + F) es un `break` puro: el programa recorre el texto y se detiene en la primera coincidencia. Por eso al pulsar «siguiente» reanuda desde donde iba, en vez de empezar de nuevo.

La validación de un formulario es `continue`: al procesar mil registros, los que vienen mal se saltan y se anotan aparte, pero el proceso no se detiene por uno malo. Un sistema que se cae porque un dato venía torcido es un sistema sin `continue`.

El mensaje «no se encontraron resultados» de cualquier buscador es, conceptualmente, el `else` de un bucle: se recorrió todo y nunca hubo `break`.

💡 ¿SABÍAS QUÉ?

En 1968, Edsger Dijkstra publicó una carta breve con un título que se volvió una de las frases más citadas de la informática: «*la instrucción goto se considera perjudicial*». El `goto` permitía saltar desde cualquier punto del programa a cualquier otro, y el resultado era un código imposible de seguir que la gente llamaba «código espagueti».

Aquella carta encendió una de las mayores peleas técnicas del siglo y ganó: de ahí nació la **programación estructurada**, la idea de que todo programa puede escribirse con solo tres cosas — secuencia, decisión y repetición — que es exactamente lo que has estado aprendiendo desde el tomo 1.

Lo interesante es que la discusión nunca se cerró del todo: `break` y `continue` son, técnicamente, saltos, y durante décadas hubo quien los consideró sospechosos por la misma razón. Ganó el sentido común, con una condición que sigue vigente hoy: **un salto es aceptable cuando hace el código más fácil de leer, y solo entonces**. Si tu `break` está enterrado en tres niveles de `if`, probablemente no lo esté haciendo.

🚩 MISIÓN DEL NIVEL 11

«Parar a tiempo»

DIFICULTAD ●●●○○ · 35 MINUTOS · ENTREGA: 4 PROGRAMAS

- 1 Busca el primer múltiplo de 17 mayor que 500 y detén el bucle al encontrarlo. Añade un contador de vueltas y muestra cuántas hiciste: ese número es lo que te ahorró el `break`.
- 2 Recorre los números del 1 al 50 e imprime solo los que **no** sean múltiplos de 3 ni de 5, usando `continue`. Escríbelo después sin `continue`, con un `if` normal, y decide cuál se lee mejor.
- 3 Escribe el control de acceso de 11.3 y modifícalo: que el número de intentos sea una constante `MAX_INTENTOS` y que al bloquearse muestre cuántos intentos usó.
- 4 Usa `for ... else` para buscar si una frase contiene alguna cifra (`if letra in "0123456789"`). Debe decir dónde apareció la primera o avisar que no hay ninguna.
- 5 **Rómpelo a propósito:** escribe el bucle peligroso del recuadro rojo (11.2), predice exactamente qué números va a imprimir antes de ejecutarlo, y córtalo con Ctrl + C.

NIVEL JEFE: escribe un detector de números primos. Pide un número y prueba si algún valor entre 2 y `numero - 1` lo divide exactamente; si encuentras uno, `break`. Usa el `else` del `for` para anunciar que es primo. Después mejóralo: **basta con probar hasta la raíz del número**. Comprueba con 97, 100 y 1 — este último te va a dar una sorpresa que tendrás que arreglar tú.

Recompensa desbloqueada: control fino. Tus bucles ya no solo repiten: deciden cuándo parar.

✓ CHECKPOINT · NIVEL 11

1. Explica la diferencia entre `break` y `continue` como si hablaras con alguien que nunca programó.
2. ¿Por qué `continue` es peligroso en un `while` y no en un `for`?
3. ¿Cuándo se ejecuta el `else` de un bucle? Escribe la regla en una sola frase.

ESTE NIVEL EN 3 IDEAS

- `break` termina el bucle entero; `continue` solo salta a la vuelta siguiente.
- `while True` es correcto siempre que exista al menos un `break` alcanzable.

- El `else` de un bucle significa «no hubo `break`»: es el mensaje de «no encontrado».

ROJO

Uso `break` y `continue` por intuición y a veces sale lo contrario.

ÁMBAR

Los distingo, pero el `else` del bucle todavía me despista.

VERDE

Escribo un `while True` con salidas claras y sé leer el `else` como «no hubo break».

Bucles anidados: ciclos dentro de ciclos

Un bucle recorre una línea. Dos bucles recorren un plano. Aquí es donde aparecen las tablas, las cuadrículas y los dibujos.

LO QUE DESBLOQUEARÁS

- 1 Escribir un bucle dentro de otro y predecir el orden exacto de las vueltas.
- 2 Construir cuadrículas, tablas y figuras de texto controlando filas y columnas.
- 3 Entender por qué anidar multiplica el trabajo, y calcular cuánto.
- 4 Saber qué bucle abandona un `break` cuando hay dos.

PALABRAS CLAVE

anidar bucle externo
bucle interno fila
columna cuadrícula
end="" salto de fila
coste
indentación doble

12.1 | La imagen mental: un reloj

LA REGLA QUE LO EXPLICA TODO

El bucle interno completa todas sus vueltas por cada vuelta del externo. Igual que el minutero da 60 vueltas por cada hora.

Un bucle dentro de otro suena complicado y es exactamente lo que ya sabes leer en un reloj. La manecilla de las horas avanza un paso; la de los minutos, entretanto, da la vuelta completa. Luego la hora avanza otro paso y los minutos vuelven a empezar desde cero.

El bucle externo es la hora. El interno son los minutos. Con esa imagen, todo lo demás del nivel se deduce solo.

reloj.py

```
1 for hora in range(3):           # bucle EXTERNO: avanza despacio
2     for minuto in range(3):     # bucle INTERNO: da la vuelta entera
3         print(f"{hora:02}:{minuto:02}", end=" ")
4     print()                     # ¡fuera del interno! salta de renglón
```

SALIDA EN PANTALLA

```
00:00 00:01 00:02
01:00 01:01 01:02
02:00 02:01 02:02
```

ORDEN REAL DE LAS 9 VUELTAS

hora = 0	min 0	min 1	min 2	salto de renglón	3 × 3 = 9 vueltas
hora = 1	min 0	min 1	min 2	salto de renglón	
hora = 2	min 0	min 1	min 2	salto de renglón	

Se lee como se escribe un texto: de izquierda a derecha, y al terminar la línea, se baja.

FIG. 12.1 – HORAS Y MINUTOS. Por cada vuelta del bucle externo, el interno se ejecuta entero y vuelve a empezar. El `print()` vacío pertenece al externo: por eso salta de renglón una vez por fila, no una vez por celda.

⚠ ERROR CLÁSICO · EL `print()` EN EL SITIO EQUIVOCADO

La indentación decide a qué bucle pertenece cada línea, y en un anidado hay **tres alturas posibles**. Mueve el `print()` vacío y obtienes tres programas distintos:

- **Dentro del interno** (8 espacios): salta de renglón en cada celda → una columna vertical larguísima.
- **Dentro del externo** (4 espacios): salta una vez por fila → la cuadrícula correcta. *Es lo que quieres.*
- **Fuera de los dos** (0 espacios): salta una sola vez al final → todo en un solo renglón interminable.

Cuando una cuadrícula «sale rara», mira primero la sangría del `print()`. Nueve de cada diez veces está ahí.

12.2 | Cuadrículas: filas y columnas

Cambia los nombres `hora` y `minuto` por `fila` y `columna` y tienes el molde de cualquier tabla del mundo. Este imprime la tabla pitagórica del 1 al 5, con anchos fijos para que las columnas queden cuadradas:

● `cuadrícula.py`

```
1 TAMANO = 5
2
3 for fila in range(1, TAMANO + 1):
4     for columna in range(1, TAMANO + 1):
5         print(f"{fila * columna:4}", end="") # ancho 4, sin salto
6     print()                                # fin de la fila
```

SALIDA EN PANTALLA

1	2	3	4	5
2	4	6	8	10
3	6	9	12	15
4	8	12	16	20
5	10	15	20	25



MICRO-RETO · 60 SEGUNDOS

Añade una fila de encabezado con los números del 1 al 5 y una línea de guiones debajo. Solo necesitas un `for` más, antes del anidado. Cuidado con la alineación: el encabezado también debe usar ancho 4.

12.3 | Dibujar con texto

TRUCO DE MULTIPLICACIÓN

`"*" * 5` produce `"*****"`
y `" " * 3` produce tres espacios. Multiplicar un texto por un número lo repite: sirve para dibujar sin bucles internos.

Este es el ejercicio más divertido del nivel y el que de verdad instala la idea, porque el resultado se ve. La clave está en darse cuenta de que **el ancho de cada fila depende del número de fila**: eso convierte una figura en una fórmula.

● `piramide.py`

```
1 ALTURA = 5
2
3 for fila in range(1, ALTURA + 1):
4     espacios = " " * (ALTURA - fila)      # cada fila lleva menos espacios
5     estrellas = "*" * (2 * fila - 1)        # y dos estrellas más que la anterior
6     print(espacios + estrellas)
```

SALIDA EN PANTALLA

```
  *
 ***
*****
*****
*****
```

Y con dos bucles anidados y un `if` puedes decidir carácter por carácter, que es literalmente como funciona una pantalla:

```

1 LADO = 5
2
3 for fila in range(LADO):
4     for columna in range(LADO):
5         if fila == 0 or fila == LADO - 1 or columna == 0 or columna == LADO - 1:
6             print("#", end="")      # está en el borde
7         else:
8             print(" ", end="")      # está en el interior
9     print()

```

SALIDA EN PANTALLA

```

#####
#  #
#  #
#  #
#####

```

12.4 | El precio de anidar

Aquí viene la advertencia seria del nivel. Cuando anidas, las vueltas **no se suman: se multiplican**. Y esa diferencia deja de ser teórica muy rápido.

Bucle externo	Bucle interno	Vueltas totales	Qué significa
10	10	100	Instantáneo.
1 000	1 000	1 000 000	Un parpadeo, todavía cómodo.
100 000	100 000	10 000 000 000	Minutos u horas. Aquí el programa deja de servir.
1 000	1 000 (× 3 niveles)	10⁹	Tres bucles anidados escalan aún peor. Evítalos salvo que sepas por qué.

La lección práctica: **si puedes resolver algo con un bucle, no uses dos**. Y si necesitas dos, procura que el interno sea lo más corto posible. Esta intuición tiene un nombre formal —*complejidad algorítmica*— y la estudiarás en serio más adelante; por ahora basta con que multipliques mentalmente antes de escribir.

Or ¿A QUÉ BUCLE AFECTA UN `break` ?

Al **más interno de los que lo contienen**, y solo a ese. Si un `break` está dentro del bucle de columnas, corta esa fila y el bucle de filas sigue tranquilamente con la fila siguiente.

Para salir de los dos a la vez, la solución de principiante —y es perfectamente válida— es una **bandera**: la levantas antes del `break` interno y, justo después del bucle interno, un `if bandera: break` corta también el externo. La solución elegante llega cuando aprendas funciones, en el próximo tomo.

🌐 EN LA VIDA REAL · LA PANTALLA QUE ESTÁS MIRANDO

Cada imagen que ves se dibuja con dos bucles anidados: para cada fila de píxeles, recorrer cada columna y decidir su color. Una pantalla Full HD son $1920 \times 1080 = \mathbf{2.073.600 \text{ píxeles}}$, y se redibuja unas 60 veces por segundo. Eso son más de 124 millones de decisiones de color por segundo, con la estructura que acabas de escribir en `marco.py`.

Cualquier hoja de cálculo recorre filas y columnas igual. Y los filtros de una foto —desenfoque, blanco y negro, contraste— son bucles anidados que visitan cada píxel y lo transforman.

Los mapas de un videojuego también: el terreno se guarda como una cuadrícula y se dibuja recorriéndola fila por fila.

💡 ¿SABÍAS QUÉ?

Los televisores antiguos, esos de tubo, dibujaban la imagen con un haz de electrones que barría la pantalla *exactamente* como tu bucle anidado: de izquierda a derecha, y al llegar al borde bajaba una línea y volvía a empezar. Ese barrido se llama *scanline*, y el momento en que el haz volvía al inicio tenía un nombre precioso: *retorno de barrido*.

Los programadores de los años ochenta descubrieron algo genial: como sabían cuánto tardaba ese retorno, podían cambiar los colores de la paleta **justo entre dos líneas**, y lograr en pantalla más colores de los que la máquina oficialmente soportaba. Muchos efectos legendarios de las consolas de la época son, en el fondo, alguien contando las vueltas del bucle interno con un cronómetro.

Las pantallas planas ya no tienen haz, pero el vocabulario se quedó: tu tarjeta gráfica sigue hablando de *líneas de barrido*. **Cuando escribes dos `for` anidados, estás usando la misma forma de pensar que dibujó todas las imágenes que has visto en tu vida.**

🚩 MISIÓN DEL NIVEL 12

«Dibujar con código»

DIFICULTAD ●●●●○ · 40 MINUTOS · ENTREGA: 5 FIGURAS

- 1 Imprime las tablas de multiplicar del 1 al 10 **completas**, una debajo de otra, con un encabezado por tabla. Son cien líneas generadas por seis líneas de código.
- 2 Dibuja: un cuadrado relleno, un triángulo alineado a la izquierda, la pirámide centrada de 12.3 y un rombo. **El rombo es el jefe de esta lista**: son dos pirámides, una al derecho y otra al revés.
- 3 Haz que la altura de la figura la pida el usuario con `input()`. Pruébala con 1, con 2 y con 20, y arregla lo que se rompa.
- 4 Escribe un tablero de ajedrez de 8×8 usando `#` y espacios. Pista: la casilla depende de si `(fila + columna) % 2` vale 0 o 1 — el operador módulo del tomo 1, otra vez trabajando.
- 5 Toma `reloj.py` y cambia el `print()` vacío de sitio a las tres alturas posibles. Ejecuta las tres versiones y describe en una línea qué pasó en cada una.

NIVEL JEFE: imprime todos los pares de números del 1 al 6 cuya suma sea exactamente 7, usando dos bucles anidados. Después optimízalo: cuando encuentres el par, ¿realmente necesitas seguir recorriendo el bucle interno? Añade el `break` y cuenta las vueltas ahorradas con un contador. Explica por qué el bucle externo **sí** debe continuar.

Recompensa desbloqueada: dos dimensiones. Ya puedes recorrer planos, no solo listas de vueltas.

✓ CHECKPOINT · NIVEL 12

1. Si el externo da 4 vueltas y el interno 6, ¿cuántas veces se ejecuta el cuerpo interno? ¿Y cuántas veces el `print()` de salto de fila?
2. ¿Qué cambia exactamente si mueves el `print()` vacío una sangría hacia la derecha?
3. ¿De qué bucle sale un `break` escrito dentro del bucle interno?

ESTE NIVEL EN 3 IDEAS

- El bucle interno se ejecuta completo por cada vuelta del externo: horas y minutos.
- La sangría decide qué pertenece a cada bucle; el `print()` vacío es lo que crea las filas.
- Anidar multiplica las vueltas: dos bucles de mil son un millón de repeticiones.

ROJO

Escribo el anidado pero no logro que la cuadrícula salga en filas.

ÁMBAR

Me sale la cuadrícula, pero las figuras con espacios todavía se me resisten.

VERDE

Predigo el orden de las vueltas y calculo el total antes de ejecutar.

Ciclos que conversan: validar, insistir y ofrecer menús

Donde tus programas dejan de romperse porque alguien escribió «hola» donde iba un número.

LO QUE DESBLOQUEARÁS

- 1 Insistir con un `while` hasta que el dato recibido sea válido.
- 2 Repetir hasta un valor centinela, sin saber cuántos datos vendrán.
- 3 Construir un menú que se repite y sale limpiamente.
- 4 Escribir mensajes que *ayudan* en vez de regañar: la parte que casi nadie enseña.

PALABRAS CLAVE

validación
rango válido
.isdigit() centinela
menú while True
match ValueError
mensaje asertivo
caso borde

13.1 | La ley del usuario impredecible

DEL NIVEL 4

`float("hola")` lanza un `ValueError` y el programa muere. Prometimos arreglarlo cuando supieras ciclos. Ha llegado el momento.

Hasta ahora tus programas pedían un dato, confiaban ciegamente en él y seguían. Eso funciona mientras *tú* seas el único que los usa. En cuanto se los pasas a otra persona, se cumple una ley sin excepciones: **alguien va a escribir algo que no esperabas**. No por maldad — por distracción, por teclado, por costumbre de escribir la coma decimal.

La respuesta no es un `if`: un `if` comprueba una vez y sigue. Lo que necesitas es **volver a preguntar tantas veces como haga falta**, y eso es un `while`.

13.2 | El patrón de validación por rango

El molde tiene una forma fija que conviene memorizar, porque la vas a escribir cientos de veces: **pide una vez fuera, y vuelve a pedir dentro mientras el dato no sirva**.

● validar_rango.py

```
1 nota = float(input("Nota (0.0 a 5.0): "))          # 1. primera petición
2
3 while nota < 0 or nota > 5:                          # 2. ¿es inválida?
4     print("Esa nota está fuera de rango. Debe ir de 0.0 a 5.0")
5     nota = float(input("Nota (0.0 a 5.0): "))      # 3. vuelve a pedir
6
7 print(f"Nota registrada: {nota:.1f}")
```

SALIDA EN PANTALLA

```
Nota (0.0 a 5.0): 7.5
Esa nota está fuera de rango. Debe ir de 0.0 a 5.0
Nota (0.0 a 5.0): -2
Esa nota está fuera de rango. Debe ir de 0.0 a 5.0
Nota (0.0 a 5.0): 4.2
Nota registrada: 4.2
```

⚠ ERROR CLÁSICO · EL `input()` QUE FALTA DENTRO

Es el error de este nivel, y produce el bucle infinito más frustrante de todos:

```
nota = float(input("Nota: "))
while nota < 0 or nota > 5:
    print("Fuera de rango") ← solo regaña; nunca vuelve a preguntar
```

La condición mira `nota`, pero dentro del bucle nadie cambia `nota`. Resultado: el mismo reproche impreso millones de veces por segundo. **Es el motor de tres tiempos del nivel 8 otra vez: falta la actualización** — solo que aquí la actualización es volver a preguntar.

13.3 | Validar el tipo antes de convertir

MÉTODOS DE TEXTO

`.isdigit()` devuelve `True` solo si todos los caracteres son dígitos. `"16"` → sí.
`"16.5"`, `"-3"`,
`"dieciséis"` → no.

Validar el rango no basta: si la persona escribe letras, el `float()` revienta antes de que tu `while` llegue a opinar. Para números enteros hay una solución limpia con lo que ya sabes: **preguntar primero, convertir después**.

● validar_tipo.py

```
1 edad_texto = input("Tu edad en años enteros: ")    # se queda como TEXTO
2
3 while not edad_texto.isdigit():                    # ¿son todos dígitos?
4     print("Necesito solo dígitos, sin letras ni comas. Ejemplo: 16")
5     edad_texto = input("Tu edad en años enteros: ")
6
7 edad = int(edad_texto)                             # ahora sí, sin riesgo
8 print(f"Perfecto, {edad} años registrados.")
```

SALIDA EN PANTALLA

Tu edad en años enteros: dieciseis
Necesito solo dígitos, sin letras ni comas. Ejemplo: 16
Tu edad en años enteros: 16,5
Necesito solo dígitos, sin letras ni comas. Ejemplo: 16
Tu edad en años enteros: 16
Perfecto, 16 años registrados.

El orden es lo único que importa aquí: **la conversión va fuera del bucle, después de que el texto pasó el examen**. Para decimales, la comprobación completa necesita herramientas del próximo tomo (`try / except`); mientras tanto, un mensaje claro que pida punto y no coma resuelve el 90 % de los casos reales.

13.4 | El centinela: repetir hasta que digan basta

¿Y si no sabes cuántos datos va a meter la persona? Reservas un valor especial que significa «terminé» — se llama **centinela** — y repites hasta que aparezca.

Elige siempre un centinela que *no pueda confundirse con un dato real*: la palabra `FIN` es buena; el número `0` casi nunca lo es.

● centinela.py

```
1 suma = 0
2 cantidad = 0
3
4 print("Escribe tus notas una por una. Escribe FIN cuando termines.")
5 dato = input("Nota: ")
6
7 while dato.upper() != "FIN":                # .upper() acepta fin, Fin, FIN
8     suma += float(dato)
9     cantidad += 1
10    dato = input("Nota: ")
11
12 if cantidad > 0:                            # ¡ojo con el caso borde!
13     print(f"Registraste {cantidad} notas. Promedio: {suma / cantidad:.2f}")
14 else:
15     print("No registraste ninguna nota.")
```

SALIDA EN PANTALLA

Escribe tus notas una por una. Escribe FIN cuando termines.

Nota: 4.0

Nota: 3.5

Nota: 5.0

Nota: fin

Registraste 3 notas. Promedio: 4.17

EL CASO BORDE QUE TE VA A MORDER

¿Qué pasa si la persona escribe `FIN` de inmediato, sin meter ninguna nota? `cantidad` vale 0 y el promedio sería `suma / 0`: un **ZeroDivisionError** en toda regla.

Por eso el `if cantidad > 0` del final no es un adorno defensivo: es el programa reconociendo que **«ningún dato» también es un resultado posible**. Cada vez que dividas por algo que un bucle contó, pregúntate qué pasa si el bucle no dio ninguna vuelta.

13.5 | El menú: la forma de todo programa interactivo

Junta `while True`, `match` del tomo 1 y un `break`, y obtienes la estructura sobre la que está construida casi cualquier aplicación de consola del mundo: **mostrar opciones, atender una, volver a mostrar**.

● cajero.py

```
1 while True:
2     print("\n1. Ver saldo   2. Consignar   3. Salir")
3     opcion = input("Elige: ")
4
5     match opcion:
6         case "1":
7             print("Tu saldo es $42.000")
8         case "2":
9             print("Consignación registrada.")
10        case "3":
11            print("Gracias por usar el cajero.")
12            break                # la ÚNICA salida del bucle
13        case _:
14            print("Esa opción no existe. Escribe 1, 2 o 3.")
```

SALIDA EN PANTALLA

1. Ver saldo 2. Consignar 3. Salir

Elige: 1

Tu saldo es \$42.000

1. Ver saldo 2. Consignar 3. Salir

Elige: 9

Esa opción no existe. Escribe 1, 2 o 3.

1. Ver saldo 2. Consignar 3. Salir

Elige: 3

Gracias por usar el cajero.

Fíjate en dos decisiones de diseño que no son casuales. Primero: **la opción se compara como texto** (`case "1"`, con comillas), porque `input()` devuelve texto — así, si alguien escribe una letra, no se rompe nada. Segundo: **el `case _` existe**, y por eso una opción inválida no tumba el programa ni lo deja en silencio: lo devuelve al menú.

13.6 | El mensaje que ayuda: asertividad en tres partes

REGLA DE ORO

El error no es culpa de quien escribe: es información que falta. Tu mensaje debe darla, no cobrarla.

Ya sabes hacer que el programa insista. Falta lo que separa un programa tolerable de uno que la gente *quiere* usar: **qué dices cuando algo sale mal**. Un buen mensaje tiene tres piezas y cabe en una línea.

1· Qué pasó

Describe el hecho, sin juzgar a nadie. «Esa fecha ya pasó», no «fecha incorrecta».

2· Qué se esperaba

Da la regla concreta. «Debe ir de 0.0 a 5.0», «solo dígitos, sin comas».

3· Qué hacer ahora

Un ejemplo válido vale más que la regla. «Ejemplo: 4.2». Ahí termina la duda.

Mensaje que estorba	Mensaje que ayuda	Qué cambió
«ERROR»	«Escribe la nota con punto, no con coma. Ejemplo: 4.2»	Dice qué hacer, no solo que algo falló.
«Dato inválido, intente de nuevo»	«La edad debe ser un número entre 1 y 120»	Da la regla exacta en vez de una etiqueta vaga.
«¡Te equivocaste!»	«No encontré esa opción. Las válidas son 1, 2 y 3»	Habla del dato, no de la persona.

«Entrada no válida (cód. 0x2F)»

«Necesito solo dígitos. Ejemplo:
16»

Usa el
idioma del
usuario,
no el del
sistema.

Un programa se juzga por cómo se comporta cuando algo sale mal, no cuando todo va bien.



REESCRIBE ESTO

Vuelve a tus programas de los niveles 10 y 11 y reescribe todos sus mensajes de error con las tres piezas. No cambies ni una línea de lógica. Léelos después en voz alta: deberían sonar como alguien ayudándote, no corrigiéndote.



EN LA VIDA REAL · TODO LO QUE INSISTE

El cajero automático es literalmente el programa de 13.5: menú, opción, volver al menú, salir. Y valida cada dato antes de aceptarlo, porque una equivocación cuesta dinero real.

El formulario que no te deja avanzar hasta que la contraseña cumpla las reglas es un bucle de validación con interfaz gráfica. Cuando te dice exactamente qué falta —«te falta un número»— alguien pensó en las tres piezas. Cuando solo dice «contraseña no válida», nadie lo pensó.

Y el «¿sigues ahí?» de las apps de video es un ciclo que lleva la cuenta de cuánto llevas sin tocar la pantalla.



¿SABÍAS QUÉ?

Hay una persona real llamada **Christopher Null** que lleva décadas siendo invisible para los sistemas informáticos. Su apellido coincide con la palabra que muchos lenguajes usan para decir «aquí no hay ningún valor», así que los programas mal escritos interpretan su apellido como un dato ausente: reservas que se pierden, formularios que se niegan a enviarse, aerolíneas que no lo encuentran.

No es un caso aislado. Las bases de datos del mundo están llenas de trampas parecidas: personas con un solo nombre, apellidos con apóstrofo, direcciones sin número. Cada una es un **caso borde** que alguien no previó porque solo probó su programa con datos parecidos a los suyos.

De ahí sale una de las frases más antiguas de la profesión: «*basura entra, basura sale*». La validación no es desconfianza hacia el usuario, es

respeto por la realidad. Y probar tu programa con datos raros —vacío, cero, negativo, enorme, con tildes— es lo que separa a quien escribe código de quien entrega software.

🚩 MISIÓN DEL NIVEL 13

«A prueba de usuarios»

DIFICULTAD ●●●●○ · 45 MINUTOS · ENTREGA: UN PROGRAMA QUE NO SE CAE

- 1 Escribe un pedidor de edad que valide tipo y rango (1 a 120), con mensajes distintos para cada caso. Pruébalo con: `hola`, `-5`, `200`, vacío y `17`.
- 2 Convierte tu informe de notas del nivel 10 en uno con centinela: sin cantidad fija, se termina escribiendo FIN. No olvides el caso de cero notas.
- 3 Construye un menú de 4 opciones para tu propio programa (calculadora, conversor de unidades, lo que quieras). Debe volver al menú después de cada opción y salir solo con la opción correspondiente.
- 4 Reescribe todos sus mensajes con las tres piezas de 13.6. Después pásale el programa a alguien que no lo haya visto y **no le expliques nada**: mira dónde se traba. Eso que viste tiene nombre profesional — prueba de usabilidad.
- 5 **Rómpelo a propósito**: quita el `input()` de dentro de un bucle de validación y observa el desastre. Después explícalo con las palabras del nivel 8.

NIVEL JEFE: añade a tu menú una opción «ayuda» y un contador de opciones inválidas. Si alguien se equivoca tres veces seguidas, muestra la ayuda automáticamente sin que la pida. Acabas de programar algo que hacen las aplicaciones buenas: **detectar que el usuario está perdido antes de que lo diga**.

Recompensa desbloqueada: programa de verdad. El tuyo ya sobrevive a otras personas.

✓ CHECKPOINT · NIVEL 13

1. ¿Por qué la validación necesita un `while` y no basta con un `if`?
2. ¿Por qué la conversión con `int()` va después del bucle y no antes?
3. ¿Qué es un centinela y qué criterio usas para elegir uno bueno?

ESTE NIVEL EN 3 IDEAS

- Validar es pedir una vez fuera del bucle y volver a pedir dentro, mientras el dato no sirva.

- Un centinela permite repetir sin saber cuántos datos vendrán; el caso de «ninguno» hay que preverlo aparte.
- `while True` + `match` + `break` es la forma estándar de un menú.

ROJO

Mi programa sigue muriendo cuando escribo algo raro.

ÁMBAR

Valido el rango, pero no el tipo, y a veces olvido volver a preguntar.

VERDE

Mi menú aguanta cualquier cosa que le escriban y explica qué hacer.

Cazar bugs de ciclos y construir algo tuyo

Los cinco errores que cometerás, cómo atraparlos en menos de cinco minutos, y el proyecto que junta los siete niveles.

🕒 LO QUE DESBLOQUEARÁS

- 1 Identificar los 5 bugs típicos de ciclos por su síntoma, sin leer todo el código.
- 2 Usar el *print espía* numerado para ver qué pasa dentro de un bucle.
- 3 Poner un límite de seguridad que impida que un `while` se des controle.
- 4 Escribir, entender y ampliar un programa completo de unas 90 líneas.

🔑 PALABRAS CLAVE

depurar print espía
traza off-by-one
bucle infinito
error lógico
límite de seguridad
caso borde
refactorizar

14.1

Los cinco bugs de ciclos, por su síntoma

DIAGNÓSTICO PRIMERO

No leas el código buscando el error. Mira el *síntoma*, elige la fila de la tabla y ve directo a la línea sospechosa. Ahorra horas.

Los errores de ciclos son sorprendentemente repetitivos. Después de estos cinco, casi todo lo demás es una variante. Aprende la tabla por el síntoma, no por la causa: el síntoma es lo primero que ves.

Lo que ves	Casi seguro es...	Dónde mirar exactamente
No para nunca / la pantalla se llena	Bucle infinito	La línea que actualiza la variable de la condición: ¿existe? ¿está dentro del bucle? ¿va en el sentido correcto? ¿hay un <code>continue</code> que la salta?
Sale una vuelta de más o de menos	Error de una unidad	Los límites del <code>range</code> o el <code><</code> contra <code><=</code> del <code>while</code> . Prueba con un caso de 2 elementos: los fallos de borde se ven al instante.
El total sale igual al último dato	Variable reiniciada	La inicialización quedó dentro del bucle. Debe estar a la misma altura que el <code>for</code> , no corrida.

Solo se procesa el primer dato

Bloque mal indentado

El cuerpo se salió del bucle. Cuenta los espacios: 4 exactos, sin mezclar tabuladores.

El resultado es plausible pero está mal

Error lógico

La condición o la fórmula. Python no puede ayudarte aquí: solo la traza a mano con datos cuyo resultado conoces.

14.2 | El print espía numerado

Ya conoces el print espía del tomo 1. Dentro de un bucle necesita una mejora imprescindible: **numerar la vuelta**. Sin ese número, cincuenta líneas de depuración idénticas no te dicen nada; con él, ves la película completa.

● espia.py

```
1 suma = 0
2 for i in range(1, 4):
3     nota = float(input(f"Nota {i}: "))
4     suma += nota
5     print(f"    [DEBUG] vuelta {i} | nota={nota} | suma={suma}")
6
7 print(f"Promedio: {suma / 3:.2f}")
```

SALIDA EN PANTALLA

```
Nota 1: 4.0
    [DEBUG] vuelta 1 | nota=4.0 | suma=4.0
Nota 2: 3.5
    [DEBUG] vuelta 2 | nota=3.5 | suma=7.5
Nota 3: 5.0
    [DEBUG] vuelta 3 | nota=5.0 | suma=12.5
Promedio: 4.17
```

Ese bloque `[DEBUG]` es tu tabla de traza, generada por el propio programa.

Cuando el bug aparece, no buscas: **lees la vuelta donde el número dejó de tener sentido**. Y cuando termines, no borres los prints: coméntalos. La semana que viene te harán falta otra vez.

🔧 CAZA EL BUG · ¿QUÉ TIENE DE MALO?

```
contador = 0
for letra in "banano":
    contador = 0
    if letra == "a":
        contador += 1
print(f"La palabra tiene {contador} letras a")
```

Imprime **0**, cuando la respuesta correcta es 2. No hay ningún mensaje de error: el programa está convencido de que hizo bien su trabajo.

Diagnóstico: la línea 3 reinicia el contador en cada vuelta, así que lo que se imprime no es la cuenta total sino *el resultado de la última vuelta*. Como «banano» termina en o, la última vuelta deja el contador en 0.

La parte traicionera: cambia la palabra a «banana» y el programa imprimirá **1** — un número que parece razonable y sigue siendo falso. *Los bugs que devuelven un valor creíble son mucho peores que los que devuelven cero.* Trázalo con las seis letras y compruébalo.

14.3 | El cinturón de seguridad

SOLO MIENTRAS DESARROLLAS

El límite de seguridad es un andamio: te protege mientras construyes. En el programa terminado sobra, porque ya sabes que el bucle termina.

Cuando estés probando un `while` del que no te fías del todo, añádele un tope artificial. Es la versión programada del cinturón: no evita el error, evita que el error te cueste la sesión de trabajo.

● cinturón.py

```
1 vueltas = 0
2 LIMITE = 1000                                # tope de seguridad mientras pruebo
3
4 while saldo < META:
5     saldo += ahorro
6     vueltas += 1
7     if vueltas > LIMITE:
8         print("[AVISO] Superé las 1000 vueltas: algo va mal. Corto.")
9         break
```

14.4 | Protocolo antipánico para ciclos

1 · Reduce el tamaño

Si falla con 100 vueltas, prueba con 3. Un bug con 3 vueltas se ve; con 100 se esconde.

2 · Imprime la vuelta

Antes de tocar nada, mete el print espía numerado. Ver es más rápido que deducir.

3 · Revisa los tres tiempos

Inicializar, comprobar, actualizar.
Recorre las tres piezas del nivel 8 en orden.

4 · Cuenta la sangría

Señala con el dedo qué líneas están dentro del bucle. ¿Son las que tú crees?

5 · Prueba los bordes

Cero elementos, un elemento, el valor exacto del límite. Ahí vive la mayoría de los bugs.

6 · Explícalo en voz alta

Línea por línea, a alguien o a un objeto. El error suele aparecer solo, a mitad de frase.

💡 ¿SABÍAS QUÉ?

El 9 de septiembre de 1947, el equipo que operaba el computador **Mark II** en la Universidad de Harvard buscaba por qué la máquina daba resultados erróneos. Al abrir un relé encontraron la causa: **una polilla** atrapada entre los contactos. La pegaron con cinta en el cuaderno de bitácora y anotaron al lado: «primer caso real de un bicho encontrado». La página se conserva, con la polilla incluida, en el museo del Smithsonian.

La palabra *bug* ya se usaba antes —hay cartas de Thomas Edison de 1878 quejándose de «bichos» en sus inventos—, así que aquel equipo no la inventó: la broma funcionaba justamente porque el término ya era conocido. Lo que sí quedó es la anécdota, ligada al nombre de **Grace Hopper**, la programadora que impulsó la idea de que el código debía escribirse en palabras parecidas al inglés y no en números — la misma idea que, décadas después, hace que tu `while saldo < meta` se lea casi como una frase.

Detalle que vale la pena guardar: el primer bug documentado no fue un error de razonamiento, sino un problema del mundo real metiéndose en la máquina. Los tuyos serán casi siempre lo contrario — y por eso puedes encontrarlos todos.

14.5 | Proyecto final: Panel de Control del Periodo

Momento de juntarlo todo. Este programa usa **los siete niveles del tomo**: `while`, `True`, `for`, `con`, `range`, los cuatro patrones, `break`, `continue`, el `else` de bucle, validación y menú. Escríbelo por partes, ejecutando después de cada una: así es como se construye software de verdad.

Parte 1 de 4 · El esqueleto

Primero la estructura vacía. Ejecuta esta parte tal cual: el menú ya debe funcionar y salir con la opción 5, aunque las demás no hagan nada todavía.

```

1 # =====
2 #  PANEL DE CONTROL DEL PERIODO  ·  Proyecto del tomo 2
3 # =====
4 import random                # librería estándar: números al azar
5
6 NOTA_MINIMA = 3.0
7 suma = 0.0                  # memoria del programa: se conserva
8 cantidad = 0               # entre una opción y la siguiente
9 mejor = 0.0
10 peor = 5.0
11
12 print("=" * 42)
13 print("      PANEL DE CONTROL DEL PERIODO")
14 print("=" * 42)
15
16 while True:
17     print("\n1. Registrar notas")
18     print("2. ¿Cuánto necesito para pasar?")
19     print("3. Tabla de multiplicar")
20     print("4. Reto: adivina el número")
21     print("5. Salir")
22     opcion = input("> ")
23
24     match opcion:
25         case "5":
26             print("Cerrando el panel. ¡Hasta la próxima!")
27             break
28         case _:
29             print("Esa opción no existe. Escribe un número del 1 al 5.")

```

Parte 2 de 4 · Registrar notas (los cuatro patrones)

Este `case` va *antes* del `case "5"`. Reúne validación de tipo, validación de rango, acumulador, contador y campeón en un solo recorrido.

```

1     case "1":
2         cuantas_texto = input("¿Cuántas notas vas a registrar? ")
3         while not cuantas_texto.isdigit() or int(cuantas_texto) == 0:
4             print("Escribe un número entero mayor que cero. Ejemplo: 4")
5             cuantas_texto = input("¿Cuántas notas vas a registrar? ")
6         cuantas = int(cuantas_texto)
7
8         suma = 0.0                # se reinicia AQUÍ a propósito:
9         cantidad = 0              # cada registro empieza de cero
10        mejor = 0.0
11        peor = 5.0
12
13        for i in range(1, cuantas + 1):
14            nota = float(input(f"Nota {i}: "))
15            while nota < 0 or nota > 5:
16                print("Fuera de rango. La nota va de 0.0 a 5.0")
17                nota = float(input(f"Nota {i}: "))
18
19            suma += nota            # acumulador
20            cantidad += 1          # contador
21            if nota > mejor:        # campeón (máximo)
22                mejor = nota
23            if nota < peor:        # campeón (mínimo)
24                peor = nota
25
26        promedio = suma / cantidad
27        estado = "APROBADO" if promedio ≥ NOTA_MINIMA else "EN RIESGO"
28        print(f"\nPromedio: {promedio:.2f}")
29        print(f"Mejor nota: {mejor:.1f}")
30        print(f"Peor nota: {peor:.1f}")
31        print(f"Estado: {estado}")

```

Parte 3 de 4 · El simulador y la tabla

La opción 2 usa un `continue` real —si no hay notas registradas, vuelve al menú— y resuelve el problema por **búsqueda por fuerza bruta**: prueba nota por nota hasta encontrar la que alcanza. No es la forma más elegante de resolverlo (con álgebra sale en una línea), pero es la más honesta y la que funciona incluso cuando no sabes despejar.

```

1     case "2":
2         if cantidad == 0:
3             print("Primero registra tus notas en la opción 1.")
4             continue                # vuelve al menú, sin salir
5
6         faltante = 0.0
7         encontrada = False           # bandera
8         while faltante ≤ 5.0:
9             if (suma + faltante) / (cantidad + 1) ≥ NOTA_MINIMA:
10                 encontrada = True
11                 break                # ya tengo la respuesta
12                 faltante += 0.1
13
14         if encontrada:
15             print(f"Con una nota de {faltante:.1f} llegas a {NOTA_MINIMA}")
16         else:
17             print("Ni con 5.0 alcanzas la meta este periodo.")
18
19     case "3":
20         tabla_texto = input("¿Tabla de qué número? ")
21         while not tabla_texto.isdigit():
22             print("Escribe solo dígitos. Ejemplo: 7")
23             tabla_texto = input("¿Tabla de qué número? ")
24         tabla = int(tabla_texto)
25
26         for i in range(1, 11):
27             print(f"{tabla} x {i:2} = {tabla * i:3}")

```

Parte 4 de 4 · El juego (y el `else` del `while`)

Aquí aparece la joya del nivel 11 aplicada de verdad: el `else` del `while` se ejecuta **solo si se agotaron los intentos sin acertar**. Si el jugador adivina, el `break` salta el `else` y no se revela el número. Toda la lógica de «perdiste» cabe ahí, sin banderas.

```

1     case "4":
2         secreto = random.randint(1, 50)
3         intentos = 0
4         print("Pensé un número del 1 al 50. Tienes 6 intentos.")
5
6         while intentos < 6:
7             intento = int(input("Tu número: "))
8             intentos += 1
9             if intento == secreto:
10                print(f"¡Correcto! Lo lograste en {intentos} intentos.")
11                break
12            elif intento < secreto:
13                print("Muy bajo.")
14            else:
15                print("Muy alto.")
16        else:
17            print(f"Se acabaron los intentos. Era {secreto}.")

```

SALIDA COMPLETA DE UNA SESIÓN

```

=====
          PANEL DE CONTROL DEL PERIODO
=====

1. Registrar notas
2. ¿Cuánto necesito para pasar?
3. Tabla de multiplicar
4. Reto: adivina el número
5. Salir
> 1
¿Cuántas notas vas a registrar? 3
Nota 1: 4.5
Nota 2: 2.8
Nota 3: 3.9

Promedio: 3.73
Mejor nota: 4.5
Peor nota: 2.8
Estado: APROBADO

1. Registrar notas
...
> 2
Con una nota de 0.8 llegas a 3.0

1. Registrar notas
...
> 9
Esa opción no existe. Escribe un número del 1 al 5.

1. Registrar notas
...
> 5
Cerrando el panel. ¡Hasta la próxima!

```

⚠️ YA TIENE UN DEFECTO · ENCUÉNTRALO Y ARRÉGLALO

En la opción 4, si el jugador escribe una letra en vez de un número, el programa se cae con un `ValueError` y pierdes toda la sesión. **Lo dejamos ahí a propósito.**

Con lo que aprendiste en el nivel 13 puedes arreglarlo sin herramientas nuevas: valida el texto con `.isdigit()` antes de convertirlo. Ese arreglo es tu primer trabajo real como programador: **encontrar el punto donde tu propio programa confía demasiado.**



RETO FINAL DEL TOMO

«Hazlo tuyo»

DIFICULTAD ●●●●● · 60 MINUTOS · ENTREGA: TU PROPIO PROGRAMA

- 1 Escribe el panel completo **a mano, parte por parte**, ejecutando después de cada una. Si escribes las cuatro de un tirón y falla, tendrás 90 líneas donde buscar.
- 2 Arregla el defecto del recuadro ámbar: valida el intento del juego antes de convertirlo.
- 3 Añade una opción 6 con un **resumen del periodo** que use los datos registrados: cuántas notas, promedio, cuántas por encima del promedio (¡otro contador!) y un mensaje distinto según el rango.
- 4 Mejora el juego: que muestre «caliente» o «frío» según lo cerca que esté el intento (pista: `abs(intento - secreto)`), y que lleve un récord de la mejor partida.
- 5 Reescribe todos los mensajes con las tres piezas del nivel 13.6 y pásaselo a alguien para que lo use sin explicaciones. Anota los tres primeros sitios donde se trabó.

JEFE FINAL: introduce a propósito, uno por uno, los cinco bugs de la tabla 14.1 en tu propio programa. Para cada uno documenta en tu cuaderno: qué síntoma vio el usuario, cómo lo detectaste y qué línea lo causaba. Ese documento es tu primer manual de depuración de ciclos, y vale más que cualquier resumen que puedas leer.

Recompensa desbloqueada: constructor de ciclos. Ya no repites código: diseñas repeticiones, las controlas y las arreglas cuando fallan.

✅ CHECKPOINT · NIVEL FINAL

1. Ves que un total sale igual al último dato introducido. ¿Cuál es tu primera sospecha y qué línea revisas?

2. ¿Por qué el print espía dentro de un bucle debe incluir el número de vuelta?
3. En el proyecto, ¿por qué el `continue` de la opción 2 no cierra el programa?

ESTE NIVEL EN 3 IDEAS

- Los bugs de ciclos son cinco y se identifican por el síntoma antes que por el código.
- El print espía numerado convierte un bucle opaco en una tabla de traza automática.
- Un programa se construye por partes, ejecutando cada una: nunca noventa líneas de un tirón.

ROJO

Cuando un ciclo falla, cambio cosas al azar hasta que funcione.

ÁMBAR

Sé usar el print espía, pero todavía no reconozco los bugs por el síntoma.

VERDE

Diagnostico por síntoma, verifico con una traza y arreglo la línea correcta.

Todo el tomo en cinco páginas

Para consultar mientras programas, no para leer de corrido. Marca esta página.

A.1 | Chuleta: los ciclos de un vistazo

Quiero...	Se escribe así
Repetir n veces exactas	<code>for i in range(n):</code>
Repetir del 1 al 10 incluido	<code>for i in range(1, 11):</code>
Contar de 2 en 2	<code>for i in range(0, 21, 2):</code>
Contar hacia atrás hasta el 0 incluido	<code>for i in range(10, -1, -1):</code>
Recorrer un texto letra por letra	<code>for letra in palabra:</code>
Repetir mientras se cumpla algo	<code>while condicion:</code> + actualizar dentro
Repetir hasta que el usuario decida	<code>while True:</code> + <code>break</code>
Insistir hasta que el dato sea válido	<code>while dato_no_sirve:</code> + volver a pedir dentro
Salir del bucle ahora mismo	<code>break</code>
Saltar solo esta vuelta	<code>continue</code>
Avisar que la búsqueda no encontró nada	<code>else:</code> alineado con el <code>for</code>
Contar cuántos cumplen	<code>total = 0</code> fuera · <code>total += 1</code> dentro de un <code>if</code>
Sumar valores	<code>suma = 0</code> fuera · <code>suma += valor</code> dentro
Buscar el mayor	<code>mejor = primer_valor</code> fuera · <code>if v > mejor:</code> <code>mejor = v</code>
Saber si pasó al menos una vez	<code>hubo = False</code> fuera · <code>hubo = True</code> dentro
Recorrer una cuadrícula	<code>for fila:</code> → <code>for col:</code> → <code>print()</code> en el externo

A.2 | Diccionario de errores de ciclos

Mensaje o síntoma	Qué significa y cómo se arregla
(nada, se queda colgado)	Bucle infinito. Corta con Ctrl + C . Revisa que la variable de la condición cambie dentro del bucle y en el sentido correcto.
IndentationError	El bloque del bucle no está alineado. Usa 4 espacios exactos y nunca mezcles tabuladores con espacios.
SyntaxError en la línea del bucle	Casi siempre faltan los dos puntos al final de <code>for ...</code> o <code>while ...</code> .
NameError después del bucle	Usas una variable creada dentro del bucle... en un bucle que dio cero vueltas, así que nunca llegó a existir.
ValueError dentro del bucle	Un <code>int()</code> o <code>float()</code> recibió algo que no era número. Valida antes de convertir (nivel 13).
ZeroDivisionError al calcular el promedio	El bucle no contó nada. Protege con <code>if cantidad > 0</code> .
TypeError al sumar	Estás acumulando texto en vez de números: falta el <code>float()</code> alrededor del <code>input()</code> .
Sale una vuelta de más o de menos	Error de una unidad. Revisa el fin del <code>range</code> (se excluye) y el <code><</code> frente al <code>≤</code> .
El total es igual al último dato	La inicialización quedó dentro del bucle. Súbela al mismo nivel que el <code>for</code> .
La cuadrícula sale en una sola columna	El <code>print()</code> vacío está dentro del bucle interno. Súbelo una sangría.

A.3

Tarjetas de repaso

CÓMO USARLAS

Tapa la respuesta, contesta en voz alta y solo entonces destapa. Recordar cuesta esfuerzo, y ese esfuerzo es exactamente lo que fija el aprendizaje.

Doce preguntas que cubren el tomo entero. Repásalas hoy, dentro de tres días y dentro de dos semanas. Si una te sale mal, vuelve al nivel correspondiente — no a leerlo: a escribir su ejemplo.

01 · Nivel 8

¿Qué tres piezas necesita todo `while`?

→ Inicializar antes, comprobar en la cabecera, actualizar dentro.

02 · Nivel 8

¿Puede un `while` dar cero vueltas?
→ Sí: si la condición ya es falsa en la primera comprobación.

03 · Nivel 9

¿Qué valores da `range(2, 9, 3)`?

→ 2, 5 y 8. El 9 marca el final y nunca se usa.

04 · Nivel 9

¿Por qué un `for` no se vuelve infinito?
→ Porque Python avanza el contador; no hay línea que olvidar.

05 · Nivel 10

Contador y acumulador: ¿en qué se diferencian?

→ Uno suma 1 (cuenta cosas); el otro suma el valor (junta cantidades).

06 · Nivel 10

¿Dónde se inicializa un acumulador?
→ Fuera del bucle. Dentro, se reinicia en cada vuelta y miente.

07 · Nivel 11

`break` frente a `continue`.

→ Break abandona el bucle entero; continue solo la vuelta actual.

08 · Nivel 11

¿Cuándo se ejecuta el `else` de un bucle?

→ Solo si el bucle terminó sin encontrarse ningún `break`.

09 · Nivel 12

Externo de 4 vueltas, interno de 6: ¿cuántas en total?

→ 24. Anidar multiplica, no suma.

10 · Nivel 12

¿De qué bucle sale un `break` del interno?

→ Solo del interno. El externo continúa con su vuelta siguiente.

11 · Nivel 13

¿Por qué validar necesita `while` y no `if`?

→ Porque hay que volver a preguntar tantas veces como haga falta.

12 · Nivel 14

El total sale igual al último dato.

¿Primera sospecha?

→ La inicialización quedó dentro del bucle.

A.4

Plantilla de traza

Cópiala en tu cuaderno. Una fila por vuelta, una columna por variable. Es la herramienta más aburrida de este tomo y la que más bugs va a cazarte.

PLANTILLA EN BLANCO

Vuelta	Variable 1	Variable 2	¿Condición?	Qué imprime
inicio			—	
1. ^a				
2. ^a				
3. ^a				
4. ^a				
salida			Falso	—

Las dos filas que casi todos olvidan son la primera y la última: el estado *antes* de entrar y la comprobación que falla y termina el ciclo. Ahí viven los errores de una unidad.

A.5 | Autoevaluación: ¿cerraste el tomo?

Marca solo lo que puedas hacer **sin mirar**. Si te quedan tres o más sin marcar, ya sabes exactamente qué repasar — y eso vale más que una nota.

✓	Nivel	Puedo...
☐	8	escribir un <code>while</code> completo y explicar sus tres piezas.
☐	8	detectar un bucle infinito leyendo el código, sin ejecutarlo.
☐	9	predecir exactamente qué valores produce cualquier <code>range()</code> .
☐	9	decidir entre <code>for</code> y <code>while</code> justificando por qué.
☐	10	escribir de memoria los cuatro patrones y combinarlos en un recorrido.
☐	11	explicar <code>break</code> , <code>continue</code> y el <code>else</code> de bucle con un ejemplo propio.
☐	12	dibujar una pirámide de texto y calcular el total de vueltas de un anidado.
☐	13	escribir un menú que no se rompa aunque le escriban cualquier cosa.
☐	13	redactar un mensaje de error con sus tres piezas.
☐	14	diagnosticar un bug de ciclo por su síntoma y confirmarlo con una traza.

LO QUE ACABAS DE APRENDER, EN PERSPECTIVA

Con variables, decisiones y ciclos tienes **las tres estructuras que bastan para escribir cualquier programa que pueda escribirse**. No es una frase motivacional: es un resultado demostrado —el teorema del programa estructurado— y significa que a partir de aquí todo lo que aprendas serán formas más cómodas de hacer cosas que ya podrías hacer.

Lo que viene en el tomo 3: las *listas*, para guardar muchos datos en una sola caja en vez de una variable por dato; y las *funciones*, para ponerle nombre a un bloque de código y reutilizarlo. Las dos encajan con los ciclos de forma casi mágica — `for nota in notas:` hará que releas el nivel 9 con otros ojos.

Pero antes de pasar página: **termina el proyecto**. Un tomo leído no cambia nada; un programa tuyo funcionando, sí.



UNA ÚLTIMA COSA

Vuelve dentro de un mes al primer ejemplo del nivel 8 — ese contador de tres vueltas que hoy te costó — y fíjate en lo obvio que te parece. Ese contraste, y no la nota de un examen, es la medida real de que aprendiste algo.